

Master Thesis

Automatic Training Set Expansion for Entity Matching Using LLM-Based Labeling and Web Retrieval

Abdullah Momin
(1869371)

July 31, 2026

Submitted to
Data and Web Science Group
Prof. Dr. Christian Bizer
University of Mannheim

Abstract

Entity matching decides whether two records describe the same real-world object, for example whether two product offers from different shops refer to the same product. Transformer-based matchers such as Ditto reach strong results but need labeled training pairs, which are expensive to obtain. This thesis asks whether an existing training set can be expanded automatically, without further human labeling, and whether the expansion improves a RoBERTa-based matcher. We compare three expansion strategies on two benchmarks (WDC Products and DBLP-Scholar): (i) string-based augmentation that rewrites existing pairs with token-level edit operations, (ii) LLM-based augmentation that selects new candidate pairs via embedding blocking and active learning and labels them with a large language model acting as a teacher, and (iii) web retrieval augmentation that gathers new product offers and paper records from the web and again lets the LLM teacher label the resulting pairs.

The outcome depends on the labeling budget, and the reasons are instructive. At the full budget, no strategy beats the baseline over three random seeds on WDC Products (best mean gain +0.016 F1, not significant), and on DBLP-Scholar only string augmentation changes the result at all: it makes it slightly worse (-0.008 F1). Even training on the union of all three additions, which overlap in zero pairs, only reproduces the best single strategy (+0.017 on WDC Products, statistically indistinguishable from string augmentation alone). But when the base training set is cut to half, the same LLM-labeled additions produce a significant gain on WDC Products (0.606 to 0.626 F1, $p=0.0002$). A string-augmentation control of matched size and class balance does not reproduce this gain (+0.000), so it comes from the teacher’s labels, not from extra volume; at a quarter of the budget, the additions rescue a training set that otherwise collapses entirely. Automatic expansion is thus neutral where the human training set already covers the common patterns, and valuable where labels are scarce. Our analysis shows why: once the expansion is forced to be entity-disjoint from the test set, the candidate pool for LLM labeling contains almost no new positive pairs (5.6% on WDC Products, 0% on DBLP-Scholar), because on a closed benchmark nearly all true matches are already part of the labeled data. The teacher itself is not the problem: its labels agree with the ground truth in 97.1% of cases (F1 0.862). Web retrieval does inject truly new positives, but the retrieved text comes from a different distribution than the benchmark records, and a follow-up experiment that rebalanced the web data and added LLM-generated hard negatives made results significantly worse rather than better. We place these findings in a detailed comparison with DistillER: Knowledge Distillation in Entity Resolution with Large Language Models (Zeakis et al. 2026) and with Labeling Training Data for Entity Matching Using Large Language Models (Steiner and Bizer 2026), whose label-replacement setting recovers over 99% of the benchmark positives

Abstract

and therefore does not face the scarcity problem: expanding a training set is structurally harder than relabeling one. Along the way, the thesis documents an entity-leakage pitfall in augmentation pipelines, quantifies how single-seed evaluations produced conclusions that did not survive seed averaging, and argues for paired significance tests on shared test sets.

Contents

Abstract	ii
1. Introduction	1
1.1. Motivation	1
1.2. Problem Statement and Research Questions	2
1.3. Contributions	2
1.4. Outline	3
2. Background and Related Work	5
2.1. Preliminaries	5
2.2. From Rule-Based Systems to Neural Entity Matching	6
2.3. Benchmarks	7
2.4. Data Augmentation for Entity Matching	7
2.5. Large Language Models for Entity Matching	8
2.6. Knowledge Distillation and LLM Teachers	8
2.7. Active Learning for Entity Matching	9
2.8. Positioning of This Thesis	10
3. Datasets	12
3.1. WDC Products	12
3.2. DBLP-Scholar	13
3.3. Serialization Format	14
3.4. Split Hygiene Analysis	14
4. Methods: Three Strategies for Training Set Expansion	16
4.1. Design Requirements for the Expanded Training Sets	16
4.2. The Student Model	17
4.3. Strategy 1: String-Based Augmentation	17
4.4. Strategy 2: LLM-Based Augmentation with Active Learning	18
4.4.1. Step 1: Embedding-Based Blocking	18
4.4.2. Step 2: Filtering the Pool	18
4.4.3. Step 3: Seed Labeling	19
4.4.4. Step 4: Active Learning Loop	20
4.5. Strategy 3: Web Retrieval Augmentation	21
4.5.1. Step 1: Entity Sampling	21
4.5.2. Step 2: Web Search	21
4.5.3. Step 3: Extraction and Labeling	22
4.5.4. Step 4: Finalization	22

4.6. Ensuring Entity-Disjoint Augmentation	22
5. Experimental Setup	24
5.1. Training Configuration	24
5.2. Metrics	24
5.3. Multi-Seed Protocol	25
5.4. Statistical Significance Testing	25
5.5. Three-Layer Evaluation Scheme	26
5.6. Measuring Teacher Label Quality	27
6. Results	28
6.1. Baselines	28
6.1.1. The WDC Products Ceiling and the Prior Shift	29
6.2. Downstream Results of the Three Strategies	30
6.3. What Multi-Seed Evaluation Changed	31
6.4. Training Set Characteristics (Layer 2)	32
6.5. Teacher Label Quality	34
6.6. Error Analysis (Layer 3)	34
6.6.1. Error Overlap Between Students	35
6.6.2. Evaluation on the Hard Test Subset	36
6.7. Summary of Findings	36
7. Second Iteration: Targeted Improvements	37
7.1. Hard-Negative Mining for DBLP-Scholar	37
7.2. Rebalanced Web Augmentation with Generated Hard Negatives	38
7.3. Training on Unions of the Augmentation Sets	39
7.4. Low-Resource Ablation	40
8. Comparison with DistillER and Steiner & Bizer	43
8.1. What Is Comparable, and What Is Not	43
8.2. Comparison with DistillER	44
8.2.1. Their Approach	44
8.2.2. Result Comparison	44
8.3. Comparison with Steiner & Bizer	45
8.3.1. Their Approach	45
8.3.2. The Pool Difference That Decides Everything	46
8.3.3. Head-to-Head Numbers	46
8.3.4. A Budget-Aware Reading of the WDC Products Gap	47
8.4. Synthesis	48
9. Discussion	50
9.1. Three Mechanisms Behind the Negative Result	50
9.2. Methodological Lessons	51
9.3. Limitations	52

Contents

10. Conclusions	53
10.1. Future Work	54
Bibliography	55
A. Teacher Prompts	59
A.1. Pair Labeling (WDC Products)	59
A.2. Pair Labeling (DBLP-Scholar)	59
A.3. Web Snippet Extraction and Labeling	59
A.4. Hard-Negative Generation and Verification	60
B. Hyperparameters and Reproducibility	61
C. Additional Tables	62
C.1. Threshold Calibration on WDC Products	62
C.2. Per-Seed F1 Values	62
C.3. Decisive-Attribute Coverage (WDC Products)	63
C.4. Candidate Pool Composition (LLM Pipeline)	63
D. Representative Error Examples	64
Ehrenwörtliche Erklärung	65

List of Tables

2.1.	The reviewed approaches, categorized along the dimensions of this chapter (citations for each approach are given in the text), with reported F1 on benchmarks shared with this thesis. DS = DBLP-Scholar, AG = Amazon-Google, WA = Walmart-Amazon, WDC = WDC Products (80% corner cases, unseen test set; small training split for this thesis). The numbers come from different training budgets and benchmark versions and are therefore only meaningful within each row's own study; Chapter 8 makes the cross-study comparison explicit.	10
3.1.	Split statistics of the two benchmarks as used in this thesis. WDC Products is the 80%-corner-case, small-training, unseen-test configuration. . . .	12
3.2.	Split hygiene of the two benchmarks, computed from the raw data. WDC Products is fully entity-disjoint; DBLP-Scholar is not, and additionally contains 186 test pairs whose serialized text is identical to a training pair.	15
6.1.	Baseline students (3-seed mean $\pm$ sample standard deviation, seeds 13/42/87).	28
6.2.	WDC Products test results (3-seed mean $\pm$ std; all runs class-weighted). No strategy differs significantly from the baseline.	30
6.3.	DBLP-Scholar test results (3-seed mean $\pm$ std). Only string augmentation differs significantly from the baseline: it is worse; the effect is nominal under Bonferroni correction.	31
6.4.	Layer 2: composition of the expanded training sets. Corner cases: hard positive = match with token-Jaccard < 0.3 ; hard negative = non-match with token-Jaccard > 0.4	32
7.1.	WDC Products: rebalanced web augmentation with generated hard negatives (3 seeds, class-weighted). *Survives Bonferroni correction (one of only two comparisons in the thesis that do, alongside the 50%-budget result of Section 7.4).	38
7.2.	Union of all three augmentation slices versus the baseline and the best single strategy (3 seeds; WDC Products runs class-weighted). The WDC Products union has the best mean of any configuration in the thesis but is statistically tied with string augmentation alone ($p = 0.214$); its nominal significance against the baseline does not survive Bonferroni correction. .	39

List of Tables

7.3. Low-resource ablation: F1 of the student trained on a reduced base set, with and without the LLM-labeled slice (3 seeds; WDC Products runs class-weighted). The “string control” row adds a string-augmented slice matched to the LLM slice in size and class balance instead (Section 7.4). *Survives Bonferroni correction. The 100% rows repeat the main results for reference.	40
8.1. DistillER (RoBERTa student, Qwen-2.5:32b teacher, ~10% of entities) versus this thesis. “Our “LLM labels” column is the <i>expansion</i> setting (GT base + LLM-labeled additions), not pure replacement; budgets and benchmark versions also differ (Section 8.1), so trends, not absolute values, are comparable.	44
8.2. Head-to-head with Steiner and Bizer (2026) (Ditto/RoBERTa student). WDC Products: same benchmark and unseen test set, different training budget. “AL (ML)” is their feature-ensemble active-learning variant, algorithmically the same selection as our <code>llm_aug</code>	46
B.1. Complete configuration of all experiments.	61
C.1. Decision-threshold calibration of the class-weighted WDC Products baseline (single seed). No selectable operating point beats the default; the oracle bound caps threshold gains at +0.018 F1.	62
C.2. Per-seed test F1 of every multi-seed run. Note the spread of the WDC Products baseline (0.028 between best and worst seed) relative to the size of the augmentation effects.	62
C.3. Share of pairs (%) in which either record mentions each decisive attribute type. All strategies broadly preserve the baseline’s coverage; none expands coverage of underrepresented attributes such as bundle contents.	63
C.4. Composition of the entity-disjoint candidate pools and the resulting teacher-label balance. For DBLP-Scholar the true-match count in the pool cannot be computed exactly (no cluster ground truth); the seeding and active-learning stages discovered none in 2,100 attempts.	63

List of Figures

3.1.	Token-Jaccard similarity of test pairs by label. Left (WDC Products): matches and non-matches overlap heavily at low similarity, so token overlap alone cannot separate the classes. Right (DBLP-Scholar): the classes are nearly separated, which anticipates both the 0.95+ F1 ceiling and the scarcity of corner cases.	13
3.2.	Whitespace-token length of serialized WDC Products training records (left and right entity), split by pair label. Most records are short (25–50 tokens); the spike at the right edge collects the long-description tail. Length is essentially uninformative about the label.	14
4.1.	Embedding cosine similarity of the filtered candidate pools (left: WDC Products, 4,722 pairs; right: DBLP-Scholar, 18,841 pairs). Nearest-neighbor blocking concentrates both pools at high similarity: plausible-looking pairs whose labels are not obvious from the surface.	19
4.2.	Share of positive teacher labels per active-learning iteration (100 pairs each). On WDC Products (left) the rate averages about 10% and declines over the iterations; on DBLP-Scholar (right) not a single positive is found among 2,000 labeled pairs. This is the positive-scarcity result of Section 4.4.2 in one picture.	21
6.1.	Confusion matrix of the class-weighted WDC Products baseline (initial single-seed run; test split, $n = 4,500$). The model over-predicts the match class (297 false positives against 124 false negatives), consistent with a decision rule calibrated to the 20% training prior and applied to an 11% test prior.	29
6.2.	Test F1 of all evaluated configurations (3-seed mean $\pm$ sample std). The dashed line marks the baseline mean. Left: WDC Products (class-weighted runs), including the union and web-v2 variants of Chapter 7. Right: DBLP-Scholar. No configuration separates from the baseline by more than the seed noise except the web-v2 variants, which are significantly worse.	30

List of Figures

6.3.	Pair-similarity distributions (token-level Jaccard) of the four training sets, split by label; top row WDC Products, bottom row DBLP-Scholar. Dotted lines mark the corner-case thresholds (hard positive < 0.3 , hard negative > 0.4). On WDC Products the match and non-match distributions overlap broadly and the corner-case band is populated; on DBLP-Scholar they are almost perfectly separated, which is the visual form of the dataset’s 3–4% corner-case share.	33
6.4.	Misclassified test pairs per run, broken down into the four error classes of Section 5.5 (computed on the seed-averaged ensemble predictions). The distribution is dominated by ambiguous variant cases on both datasets, and the profile barely changes across training sets: augmentation shifted error counts, not error kinds.	35
7.1.	WDC Products: effect of the LLM-labeled additions as a function of the base training budget (3-seed mean $\pm$ std; annotations give the mean difference vs. the base). The same 2,100-pair slice that ties the baseline at the full budget helps significantly at half the budget and rescues an otherwise untrainable set at a quarter. The orange bar at the 50% budget is the volume control: a string-augmented slice of matched size and class balance, which gains nothing over the base.	41
8.1.	Students trained on ground-truth labels versus LLM-teacher labels across the three studies (RoBERTa/Ditto students; annotations give the delta). Distiller numbers from their Table 8 ($\sim 10\%$ of entities); Steiner & Bizer numbers from their Table 1 (AL-ML variant); our bars are the expansion setting with 3-seed error bars. Budgets and benchmark versions differ between studies (Section 8.1), so the figure compares directions, not absolute levels: teacher labels help only in Distiller’s most data-starved product case (Walmart-Amazon) and are roughly neutral or slightly harmful everywhere else.	47
8.2.	WDC Products F1 as a function of the number of training pairs (log scale). Purple: the machine-label learning curve of Steiner and Bizer (2026) , Active learning (ML) variant, digitized from their Figure 4 with the endpoint pinned to their Table 1 value (70.65) and the benchmark line at 71.94; read-off precision is about ± 0.5 –1 F1 per point. The star marks their large-split human-label benchmark. Gray/green with error bars: this thesis’s configurations at their actual training sizes, including the reduced-budget points of Section 7.4. Our human-label baseline at 2,500 pairs lies on their curve; at full budget none of our expansions moves meaningfully up it, while in the reduced-budget region the LLM additions produce visible gains. Growing an adequate training set with expandable pairs is not equivalent to growing it with benchmark-quality labeled pairs, but growing an inadequate one helps.	48

1. Introduction

1.1. Motivation

A large part of the value of data comes from combining it. Price comparison portals combine product offers from thousands of shops, bibliographic services combine publication records from different digital libraries, and companies merge customer databases after acquisitions. In all of these settings the same question appears again and again: do these two records describe the same real-world thing? This question is called entity matching (also record linkage or entity resolution), and it has been studied for over sixty years (Newcombe et al. 1959; Fellegi and Sunter 1969; Christen 2012).

The current state of the art treats entity matching as a binary classification problem over record pairs and solves it with pre-trained transformer language models. Ditto (Li et al. 2020), which fine-tunes models such as RoBERTa (Liu et al. 2019) on serialized record pairs, remains a strong and widely used reference point. The catch is the training data. Fine-tuning needs thousands of labeled pairs, and labeling pairs is slow, boring, and expensive work that requires some domain knowledge: deciding whether two laptop offers describe the same configuration means reading model numbers, memory sizes, and bundle contents carefully. The cost of building training sets, not the model architecture, is today the main obstacle to using neural entity matching in practice (Barlaug and Gulla 2021).

Two recent developments promise a way out. First, large language models (LLMs) have become good at entity matching themselves when prompted directly (Narayan et al. 2022; Peeters and Bizer 2023; Peeters et al. 2025), which raises the possibility of using an LLM as a cheap annotator: let the LLM label pairs, and train a small, fast model on those labels. This is knowledge distillation in the classic sense of Hinton et al. (2015), with the LLM as the teacher and the small model as the student. Two works pursue this direction and form the main points of comparison for this thesis: DistillER (Zeakis et al. 2026) distills LLM labels into small student models across eight benchmarks, and Steiner and Bizer (2026) replace the human labels of existing benchmarks with LLM labels and measure how much downstream quality is lost. Second, the web itself is a source of additional records: for a product in the training set, other shops sell the same product and describe it differently, and a web search API can retrieve those descriptions.

This thesis combines both ideas and asks a question that sounds similar to prior work but differs in one important way. DistillER and Steiner and Bizer (2026) study *replacement*: they take the pairs of an existing benchmark and relabel them with an LLM teacher. We study *expansion*: we keep the existing human-labeled training set and try to add *new* pairs on top of it: pairs the benchmark does not contain, found either by blocking over the entity tables or by retrieving new records from the web, and

labeled by the LLM teacher. Expansion is the scenario a practitioner actually faces: they already have some labeled data and want to know whether automatic methods can enlarge it profitably. As we will see, this seemingly small difference in framing has large consequences.

1.2. Problem Statement and Research Questions

We are given a training set $T = \{(l_i, r_i, y_i)\}$ of record pairs with binary labels ($y = 1$: match, $y = 0$: non-match), a validation set, and a held-out test set from the same benchmark. The task is to construct an expanded training set $T' \supseteq T$ automatically, without any additional human labels, such that a student model fine-tuned on T' achieves higher F1 on the test set than the same model fine-tuned on T . The expansion must respect a hygiene constraint: it may not use any entity that only appears in the validation or test split, since otherwise the student would see held-out entity descriptions during training.

The thesis answers the following research questions:

- **RQ1:** Does simple string-based augmentation, which creates modified copies of existing pairs, improve the downstream F1 of a RoBERTa student?
- **RQ2:** Does LLM-based augmentation, which selects new pairs from a blocked candidate pool via active learning and labels them with an LLM teacher, improve downstream F1?
- **RQ3:** Does web retrieval augmentation, which obtains truly new records from outside the benchmark and pairs them with training entities, improve downstream F1?
- **RQ4:** How do the expanded training sets compare in terms of label ratio, corner-case proportion, attribute coverage, and mutual overlap, and how close do they come to the design targets set for them?
- **RQ5:** How do the results relate to the label-replacement settings of DistillER (Zeakis et al. 2026) and Steiner and Bizer (2026), and what explains the differences?

1.3. Contributions

The main contributions of this thesis are the following:

1. **A controlled comparison of three automatic expansion strategies** (string-based, LLM-based with active learning, and web-retrieval-based) on two benchmarks with different characteristics (WDC Products, an e-commerce benchmark with abundant hard cases, and DBLP-Scholar, a bibliographic benchmark near the performance ceiling). All strategies are evaluated with the same student model, the same three-layer evaluation scheme, three random seeds, and paired significance tests.

1. Introduction

2. **A negative result with an explanation, and its boundary.** At the full labeling budget, no expansion strategy robustly improves the student. We trace this to three separate mechanisms: positive scarcity in entity-disjoint candidate pools (the benchmark already contains nearly all true matches among its entities), a text-distribution mismatch between web-retrieved records and benchmark records, and a performance ceiling on DBLP-Scholar that leaves no room for improvement. We consider the first mechanism the central finding of the thesis: *expanding* a closed benchmark honestly is structurally harder than *relabeling* it, because the discoverable new pairs are almost all negatives. A low-resource ablation then locates the boundary of the negative result: when the base training set is halved, the same LLM-labeled additions yield a significant gain (+0.019 F1, $p = 0.0002$) that a volume-matched string-augmentation control does not reproduce. The value of expansion is a function of label scarcity, not a fixed property of the data, and where it appears it is carried by the teacher’s labels rather than by data volume.
3. **A quantified teacher-quality analysis.** Using the cluster ground truth of WDC Products, we measure the LLM teacher’s labeling quality exactly (accuracy 0.971, F1 0.862, 2.9% label noise) and show that teacher noise is not the bottleneck of LLM-based expansion.
4. **A methodological case study.** The first version of our LLM and web pipelines leaked held-out entities into the training data; we document the leak, the fix, and its effects. Similarly, several single-seed conclusions (including one apparently significant improvement) did not survive averaging over three seeds. Both episodes are reported openly because they are easy mistakes to make and their effects were large enough to flip conclusions.
5. **An in-depth comparison with two related systems,** DistillER and [Steiner and Bizer \(2026\)](#), that reconciles apparently contradictory numbers by making training budgets and test-set versions explicit, and positions augmentation and replacement as two distinct regimes of LLM-teacher entity matching.

All code, pipelines, and analysis scripts used in this thesis are available at the accompanying GitHub repository.¹

1.4. Outline

Chapter 2 introduces the entity matching problem and reviews related work on neural matchers, data augmentation, LLMs for data integration, knowledge distillation, and active learning. Chapter 3 describes the two benchmarks, their preprocessing, and a split-hygiene analysis whose results matter later. Chapter 4 presents the student model and the three expansion pipelines in detail, including the entity-disjointness policy. Chapter 5 defines the experimental setup: training configuration, metrics, the multi-seed protocol,

¹<https://github.com/abdmomin/thesis-project>

1. Introduction

significance testing, and the three-layer evaluation scheme. Chapter 6 reports the main results. Chapter 7 covers the second experimental iteration: hard-negative mining for DBLP-Scholar, a rebalanced web augmentation variant with generated hard negatives, training on the union of all augmentation sets, and a low-resource ablation that yields the thesis’s one clear positive result. Chapter 8 compares our findings with DistillER and [Steiner and Bizer \(2026\)](#). Chapter 9 discusses the mechanisms behind the results and the limitations of the study, and Chapter 10 concludes.

2. Background and Related Work

This chapter first fixes terminology and describes the standard entity matching pipeline (Section 2.1). It then reviews the lines of work this thesis builds on: neural entity matching (Section 2.2), data augmentation (Section 2.4), large language models for entity matching (Section 2.5), knowledge distillation with LLM teachers (Section 2.6), and active learning (Section 2.7). To keep the review comparable across very different systems, we discuss each approach along four recurring dimensions: the matching model it uses, where its training labels come from, what it does to the training data (use it as given, perturb it, relabel it, or expand it), and the labeling budget at which it was evaluated. Section 2.8 collects the reviewed approaches in an overview table along these dimensions, together with their reported F1 scores, and states the gap this thesis addresses. Chapter 8 comes back to these numbers at the end of the thesis and compares them with our own results in detail.

2.1. Preliminaries

Definition 1 (Entity Matching) *Given two collections of records A and B , where each record describes a real-world entity through a set of attribute values, entity matching is the task of finding all pairs $(a, b) \in A \times B$ such that a and b refer to the same real-world entity.*

In practice the task is solved in two stages (Christen 2012; Papadakis et al. 2020). The first stage, *blocking*, cheaply discards the vast majority of the $|A| \times |B|$ possible pairs so that only plausible candidates remain; without blocking, even the moderately sized datasets used in this thesis would produce up to 168 million pairs. The second stage, *matching*, applies a more expensive classifier to each surviving candidate pair and outputs a binary decision. This thesis is concerned with the matching stage: all benchmarks used here provide fixed sets of candidate pairs with human labels, and the matcher is trained and evaluated on those pairs. Blocking still appears in one place, though: our LLM-based expansion pipeline uses embedding-based blocking to generate its own candidate pairs (Section 4.4).

Matching quality is measured on the positive (match) class. With tp , fp , and fn denoting true positives, false positives, and false negatives, precision is $P = tp/(tp + fp)$, recall is $R = tp/(tp + fn)$, and the F1 score is their harmonic mean, $F_1 = 2PR/(P + R)$. F1 on the match class is the standard headline metric in entity matching because the class distribution is heavily skewed toward non-matches (Köpcke et al. 2010), and we follow that convention throughout.

2. Background and Related Work

Two properties of a benchmark deserve attention because they will play a large role later. First, a benchmark is *entity-disjoint* if no entity that appears in the training split also appears in the validation or test split. Entity-disjoint test sets measure generalization to unseen entities; non-disjoint test sets partly measure memorization. Second, benchmarks differ in their share of *corner cases*: pairs that are hard because surface similarity and true label disagree: highly similar non-matches (for example, the 500 GB and 1 TB variants of the same SSD) and dissimilar matches (the same product described with completely different words).

2.2. From Rule-Based Systems to Neural Entity Matching

Entity matching predates machine learning. Newcombe et al. (1959) already linked vital records automatically in 1959, and Fellegi and Sunter (1969) gave the problem its classic probabilistic formulation, modeling the odds that agreement patterns between record fields arise from a true match versus a coincidence. Decades of subsequent work refined similarity functions, blocking schemes, and rule-based decision models; Elmagarmid et al. (2007) and Christen (2012) survey this period. Systems such as Magellan (Konda et al. 2016) packaged the resulting best practices (feature engineering over string similarity measures, followed by a conventional classifier such as a random forest) into a usable toolchain, and this feature-based approach remains a strong baseline on structured data.

Deep learning entered the field with DeepER (Ebraheem et al. 2018) and DeepMatcher (Mudgal et al. 2018), which replaced hand-crafted similarity features with learned representations built from word embeddings and recurrent networks. DeepMatcher’s design-space study showed that the gains of deep models over Magellan were largest on textual and dirty data, exactly the settings where hand-crafted features struggle, and small on clean, structured data. The reported numbers make the step concrete: on DBLP-Scholar, Magellan reaches an F1 of 0.923 and DeepMatcher 0.947, and on the harder Amazon-Google product benchmark the move from feature engineering to learned representations lifts F1 from 0.491 to 0.693 (Mudgal et al. 2018). The next step followed the general path of NLP: pre-trained transformers (Vaswani et al. 2017; Devlin et al. 2019). Brunner and Stockinger (2020) showed that fine-tuning BERT-family models on serialized record pairs beats previous architectures, and Ditto (Li et al. 2020) turned this into the reference architecture that this thesis also uses. Ditto serializes each record into a tagged string (`COL attribute VAL value ...`), concatenates the two records of a pair with a separator token, and fine-tunes a pre-trained language model such as RoBERTa (Liu et al. 2019) with a binary classification head. On the benchmarks above, Ditto pushed F1 further, to 0.956 on DBLP-Scholar and 0.756 on Amazon-Google, and its product-benchmark results (0.893 on Abt-Buy, 0.868 on Walmart-Amazon, against 0.990 on the clean DBLP-ACM) already show the pattern that products are the hard domain (Li et al. 2020). Ditto also introduced three optimizations (domain knowledge injection, summarization of long values, and data augmentation), of which the last is a direct ancestor of our first expansion strategy. Barlaug and Gulla (2021) give a survey of this generation of matchers.

2. Background and Related Work

The important background fact for this thesis is that transformer matchers are data-hungry. Ditto’s reported advantage over DeepMatcher grows with training set size, and on small training sets fine-tuning is unstable: as we will see in Chapter 6, the same model trained on the same 2,500 pairs with different random seeds can vary by more than 0.02 F1. This data hunger motivates the entire premise of the thesis, and the seed variance shapes its evaluation protocol.

2.3. Benchmarks

Most published entity matching results use the benchmark collection assembled around Magellan and DeepMatcher (Konda et al. 2016; Mudgal et al. 2018), which includes the bibliographic DBLP-Scholar dataset used in this thesis, and the WDC product corpora (Primpeli et al. 2019). Köpcke et al. (2010) established DBLP-Scholar as a standard task: matching computer science publication records from DBLP against records crawled from Google Scholar, where Scholar records are notoriously noisy (truncated author lists, garbled venues, missing years).

WDC Products (Peeters et al. 2024) is a newer benchmark built from product offers on the web that carry schema.org annotations, which supply cluster identifiers as ground truth. What sets it apart is that difficulty is a controlled dimension: the benchmark comes in variants with a chosen share of corner cases (we use the hardest, “80% corner cases” variant), with seen or unseen test entities (we use unseen, i.e. the entity-disjoint variant), and with three training sizes (we use small). This controllability is why WDC Products is our primary benchmark: it was explicitly designed to punish matchers that rely on surface similarity, which is the failure mode that data augmentation is supposed to address. It is also clearly harder than the older product benchmarks: a RoBERTa-based matcher trained on the large split of the 80%-corner-case, unseen-test configuration reaches an F1 of about 0.72 (Peeters et al. 2024; Steiner and Bizer 2026), compared to Ditto’s 0.87–0.89 on Walmart-Amazon and Abt-Buy.

2.4. Data Augmentation for Entity Matching

Data augmentation enlarges a training set by transforming existing examples in label-preserving ways. In NLP, simple token-level operations such as synonym replacement, random insertion, swap, and deletion can help small-data text classification (Wei and Zou 2019); Shorten et al. (2021) survey the area. Ditto (Li et al. 2020) adapted this idea to entity matching with operators that respect the structure of serialized records: deleting spans of value tokens, shuffling token spans, dropping whole attributes, and swapping the two records of a pair. The intuition is that a matcher should not change its decision because a word was dropped or an attribute reordered, so training on such perturbed copies teaches invariance.

In Ditto’s own ablations, augmentation is worth on the order of one F1 point on average, with larger gains on small training sets (Li et al. 2020). Rotom (Miao et al. 2021) took augmentation a step further with a meta-learning framework that learns

2. Background and Related Work

which augmentation policies to apply and how to weight the generated examples, and reports that this beats fixed policies on entity matching among other tasks, again with the largest margins in low-resource settings. Our string-based strategy (Section 4.3) deliberately stays with Ditto’s fixed operators: it is the simple, cheap, LLM-free baseline against which the two more elaborate strategies must justify their cost.

A shared limitation of all perturbation-based augmentation is that it cannot create truly new information. A perturbed copy of a pair contains the same entities, the same decisive attributes, and the same label; the model sees more variation but no new facts. The two other strategies in this thesis are attempts to break this limitation, by pairing entities in new combinations (LLM-based expansion) and by importing entirely new records from outside the benchmark (web retrieval).

2.5. Large Language Models for Entity Matching

Narayan et al. (2022) first showed that large autoregressive language models can perform data integration tasks, including entity matching, from prompts with few or no examples. Peeters and Bizer (2023) evaluated ChatGPT on product matching and found it competitive with fine-tuned baselines that had seen thousands of labeled pairs, despite receiving no or only a handful of examples in the prompt; the extended study of Peeters et al. (2025) systematically compared prompt designs, model families, and fine-tuning across benchmarks, and found that hosted frontier models match or exceed a fine-tuned RoBERTa on many datasets, with the practical drawbacks of cost and latency per pair.

These results changed the economics of the field. If an LLM can label a pair for a fraction of a cent at near-human quality, human annotation stops being the only way to create training data. Two consequences are being explored in the literature. The first is to use the LLM directly as the matcher, which is accurate but expensive at scale, since every candidate pair triggers an API call at inference time. The second is to use the LLM as an annotator that creates training data for a small student model, which is then cheap at inference time. This second route is the one this thesis takes, and it leads directly to knowledge distillation.

2.6. Knowledge Distillation and LLM Teachers

Knowledge distillation (Hinton et al. 2015) originally described training a small student network to imitate the output distribution of a large teacher network; DistilBERT (Sanh et al. 2019) is a well-known application to pre-trained language models, and Gou et al. (2021) survey the many variants. In the looser sense used in recent entity matching work, and in this thesis, distillation means training a student on *labels produced by a teacher* rather than on ground-truth labels. The teacher is a prompted LLM, the student a fine-tuned compact model, and the interesting question is how much quality survives the noisy teacher.

Two works define the state of the art here, and Chapter 8 compares against both in detail; we summarize them briefly. Distiller (Zeakis et al. 2026) distills the labels

2. Background and Related Work

of a Qwen-2.5 32B teacher into small students (RoBERTa among them) across eight standard benchmarks in a deliberately low-resource setting where only about 10% of entities are available for training-pair construction. Their central result is encouraging for distillation: the RoBERTa student reaches an average F1 of 0.69 with teacher labels versus 0.66 with ground-truth labels over the eight datasets, so noisy LLM labels were on average no worse than human labels for the student, and on some product datasets they were better. The spread across datasets is large, though: on their DBLP-Scholar variant the student loses a little with teacher labels (0.86 versus 0.89), while on the data-starved Walmart-Amazon benchmark it gains a lot (0.57 versus 0.44). They also study unsupervised selection of pairs to label, finding similarity-ranking-based selection that roughly maintains a 3:1 negative-to-positive ratio to be the most reliable.

Steiner and Bizer (2026) ask the replacement question directly: if we relabel the training pairs of existing benchmarks with an LLM teacher (GPT-based, among others), how much downstream F1 does a student like Ditto lose compared to training on the original human labels? Their answer is: little. Across benchmarks including WDC Products and DBLP-Scholar, students trained on machine labels come close to students trained on human labels; on WDC Products, for example, their active-learning selection variant reaches 0.707 F1 against a human-label benchmark of 0.719, and on DBLP-Scholar 0.938 against 0.956. They also compare several strategies for *choosing* which pairs to label, including an active-learning variant with an ensemble of feature-based classifiers; that variant is the direct blueprint for our LLM-based expansion pipeline (Section 4.4). A crucial detail, easy to overlook and central to this thesis, is that their candidate pools are built from the benchmark’s own pair universe and recover 99.56–99.92% of the original positive pairs. Their teacher relabels pairs the benchmark already knew about. Whether an LLM teacher can find *new* matches beyond the benchmark’s own pairs is exactly the question their setup does not pose, and ours does.

2.7. Active Learning for Entity Matching

Active learning selects the most informative examples to label next, instead of labeling at random (Settles 2009). In entity matching, it has a long history because labeling budgets are the binding constraint; Meduri et al. (2020) benchmark many combinations of learners and selection strategies, and Kasai et al. (2019) combine transfer learning with active learning for deep matchers under tight budgets. Primpeli and Bizer (2021) exploit the graph structure of match decisions to improve selection. The selection principle used in this thesis is committee-based uncertainty (Settles 2009): train several diverse classifiers on the labeled pool, and query the pairs on which the committee disagrees most. Following Steiner and Bizer (2026), our committee consists of a random forest (Breiman 2001), XGBoost (Chen and Guestrin 2016), gradient boosting (Friedman 2001), and logistic regression, all over classic string-similarity features: a deliberately cheap committee, since its only job is to point the expensive LLM teacher at informative pairs.

2.8. Positioning of This Thesis

Table 2.1 organizes the approaches reviewed in this chapter along four dimensions: the *matching model* (feature-based classifier, neural matcher, pre-trained transformer, or prompted LLM), the *label source* (human annotation or an LLM teacher), the *data operation* (whether training pairs are used as given, perturbed, relabeled, or newly added), and the *reported F1* on the benchmarks that overlap with this thesis. The F1 values are not directly comparable across rows, because training budgets and benchmark versions differ between the studies, but they show the shape of the field.

Approach	Matcher	Label source	Data operation	Reported F1
Magellan	feature ML	human	as given	0.923 DS, 0.491 AG
DeepMatcher	RNN	human	as given	0.947 DS, 0.693 AG
Ditto	transformer	human	perturb	0.956 DS, 0.756 AG
Rotom	transformer	human	perturb (learned)	beats fixed policies
Prompted LLMs	LLM	zero-/few-shot	none	$\approx$ fine-tuned student
Distiller	RoBERTa student	LLM teacher	relabel ($\sim 10\%$)	0.86 DS, 0.57 WA
Steiner & Bizer	Ditto student	LLM teacher	relabel (full scale)	0.938 DS, 0.707 WDC
This thesis	RoBERTa student	human + LLM teacher	expand	0.957 DS, 0.661 WDC

Table 2.1.: The reviewed approaches, categorized along the dimensions of this chapter (citations for each approach are given in the text), with reported F1 on benchmarks shared with this thesis. DS = DBLP-Scholar, AG = Amazon-Google, WA = Walmart-Amazon, WDC = WDC Products (80% corner cases, unseen test set; small training split for this thesis). The numbers come from different training budgets and benchmark versions and are therefore only meaningful within each row’s own study; Chapter 8 makes the cross-study comparison explicit.

Read along the first dimension, the table shows that the largest quality jumps came from changing the matching model: on DBLP-Scholar, F1 rises from 0.923 (Magellan) over 0.947 (DeepMatcher) to 0.956 (Ditto), and on Amazon-Google from 0.491 over 0.693 to 0.756. Everything after Ditto competes within a few F1 points on the same student architectures, which is why recent work has shifted its attention from the model to the training data. Read along the label-source and data-operation dimensions, the recent LLM-teacher works all sit in the *relabel* rows: Distiller and Steiner and Bizer (2026) let the teacher label pairs the benchmark already contains, and both find that students trained on those machine labels stay close to students trained on human labels (0.69 versus 0.66 average F1 for Distiller’s RoBERTa student; 0.707 versus 0.719 on WDC Products for Steiner and Bizer 2026). Perturbation-based augmentation (Li et al. 2020; Miao et al. 2021) occupies the *perturb* rows and adds robustness but no new information. The *expand* row, adding new labeled pairs to an existing human-labeled set, is empty before this thesis.

Against this backdrop, the gap this thesis addresses can be stated precisely. Prior work has shown that LLM teachers can *replace* human annotators on existing benchmark pairs

2. Background and Related Work

at small quality loss (Steiner and Bizer 2026; Zeakis et al. 2026), and that perturbation-based augmentation can help transformer matchers (Li et al. 2020; Miao et al. 2021). What has not been studied is the *expansion* setting: keeping the human training set and adding new, automatically discovered, LLM-labeled pairs on top, either from the benchmark’s own entity tables via blocking, or from outside the benchmark via web retrieval. Expansion differs from replacement in what the candidate pool can contain, and it differs from perturbation in that it can, in principle, add new information. The thesis evaluates both new routes against the perturbation baseline under identical conditions, imposes the entity-disjointness hygiene that the setting requires, and explains the results through the composition of the candidate pools rather than only through downstream F1. At the end of the thesis, Chapter 8 returns to the numbers in Table 2.1 and works out, budget by budget, how our results relate to them.

3. Datasets

We evaluate on two benchmarks that were chosen to be as different as possible in the properties that matter for augmentation: WDC Products, an e-commerce benchmark deliberately built around hard cases with plenty of headroom for improvement, and DBLP-Scholar, a bibliographic benchmark on which transformer matchers already operate near their ceiling. This chapter describes both datasets, the shared serialization format, and a split-hygiene analysis that turned out to be important for interpreting the results.

3.1. WDC Products

WDC Products (Peeters et al. 2024) contains product offers extracted from web pages with schema.org annotations. Offers that carry the same product identifier (e.g. a GTIN or MPN) form clusters, and these cluster memberships serve as ground truth: two offers match if and only if they belong to the same cluster. Each offer has the attributes **brand**, **title**, **description**, **price**, and **priceCurrency**. The benchmark spans many product categories, with a concentration of electronics and computer hardware, a domain where near-identical product variants (storage sizes, colors, bundled accessories) make matching truly difficult.

Dataset	Split	Pairs	Matches	Non-matches	% Positive
WDC Products	train	2,500	500	2,000	20.0
	valid	2,500	500	2,000	20.0
	test	4,500	500	4,000	11.1
DBLP-Scholar	train	17,223	3,207	14,016	18.6
	valid	5,742	1,070	4,672	18.6
	test	5,742	1,070	4,672	18.6

Table 3.1.: Split statistics of the two benchmarks as used in this thesis. WDC Products is the 80%-corner-case, small-training, unseen-test configuration.

We use the configuration that is hardest and cleanest at the same time: the **80% corner-case** variant (80% of the pairs are hard positives or hard negatives by the benchmark’s own construction), the **unseen** test set (test entities are disjoint from training entities), and the **small** training size. Table 3.1 shows the split statistics. Two numbers deserve emphasis. First, the training set is small: 2,500 pairs over roughly 1,000 distinct entities; this is the low-resource setting in which augmentation should matter

3. Datasets

most. Second, the positive rate differs between splits: 20.0% of training and validation pairs are matches, but only 11.1% of test pairs are. This *prior shift* is a property of the published benchmark, not of our processing, and Section 6.1.1 shows that it acts as a hidden ceiling on test F1.

3.2. DBLP-Scholar

DBLP-Scholar (Köpcke et al. 2010) matches publication records from the DBLP bibliography (2,616 records in the version used here) against records crawled from Google Scholar (64,263 records). Each record has the attributes `title`, `authors`, `venue`, and `year`. The Scholar side is noisy in characteristic ways: author lists are abbreviated or truncated, venue strings are inconsistent, years are missing. We use the standard split from the Magellan/DeepMatcher benchmark collection (Mudgal et al. 2018), with 17,223 training, 5,742 validation, and 5,742 test pairs (Table 3.1).

DBLP-Scholar plays the role of a contrast dataset in this thesis. Fine-tuned transformers reach F1 around 0.95 on it, so there is little headroom; its pairs are mostly either clearly matching or clearly different (few corner cases, as quantified in Section 6.4); and its split has hygiene issues that the next section makes precise. Figure 3.1 makes the contrast between the two benchmarks visible before any model is trained: on the WDC Products test split, the similarity distributions of matches and non-matches lie almost on top of each other at low token overlap (surface similarity is nearly uninformative, by the benchmark’s corner-case design), whereas on DBLP-Scholar the two classes separate cleanly, with only a thin overlap band. If augmentation helps anywhere, it should be on WDC Products; DBLP-Scholar tests whether augmentation at least does no harm near the ceiling.

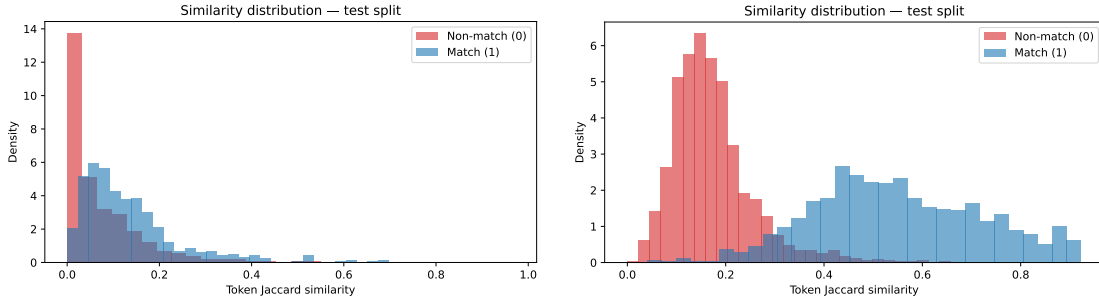


Figure 3.1.: Token-Jaccard similarity of test pairs by label. Left (WDC Products): matches and non-matches overlap heavily at low similarity, so token overlap alone cannot separate the classes. Right (DBLP-Scholar): the classes are nearly separated, which anticipates both the 0.95+ F1 ceiling and the scarcity of corner cases.

3.3. Serialization Format

Both datasets are converted into Ditto’s serialization format (Li et al. 2020). Each record becomes a string of the form

```
COL title VAL Querying and Mining Data Streams ... COL venue VAL SIGMOD
COL year VAL 2002
```

and a pair is stored as the two serialized records plus a tab-separated binary label. WDC Products records serialize the attributes **brand**, **title**, **description**, **price**, and **priceCurrency**; DBLP-Scholar records serialize **title**, **authors**, **venue**, and **year**. Missing values become empty strings after the VAL marker. All training files produced in this thesis (the original splits and every augmented variant) use exactly this format, so any of them can be fed to the same training script unchanged.

Serialized records must fit the student’s input window, and Figure 3.2 shows they mostly do: WDC Products records concentrate at 25–50 whitespace tokens, with a long tail from verbose product descriptions that accumulates at the length cap. Since a pair encodes both records, the student’s limit of 256 subword tokens per pair (Section 4.2) truncates only the tail of that tail; the identifying attributes (brand, title, and the head of the description) come first in the serialization and survive truncation.

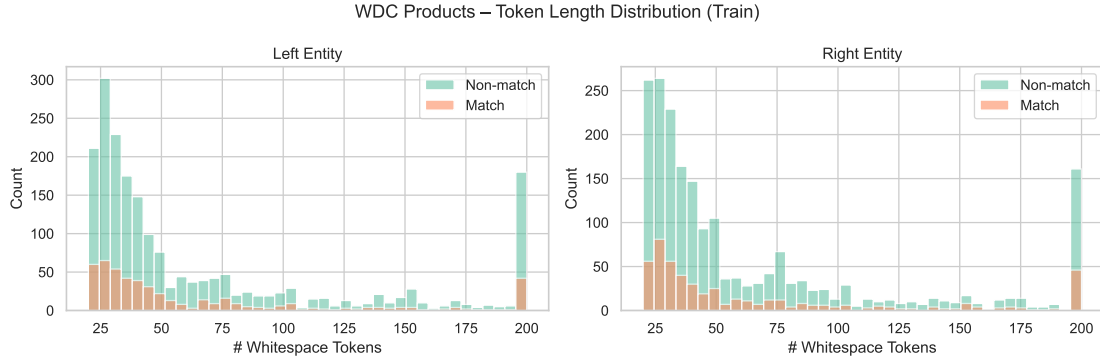


Figure 3.2.: Whitespace-token length of serialized WDC Products training records (left and right entity), split by pair label. Most records are short (25–50 tokens); the spike at the right edge collects the long-description tail. Length is essentially uninformative about the label.

3.4. Split Hygiene Analysis

Because the whole point of this thesis is to add training pairs, we need to know precisely what the existing splits do and do not share. We verified three properties directly on the raw data; the results are summarized in Table 3.2 and referenced repeatedly in later chapters.

3. Datasets

	WDC Products	DBLP-Scholar
Test entities also in train (left)	0 (0.0%)	1,588 / 1,708 (93.0%)
Test entities also in train (right)	0 (0.0%)	2,502 / 3,938 (63.5%)
Shared train–test pairs (by record ID)	0	0
Shared train–test pairs (by exact text)	0	186 (3.2% of test)
Positive rate train / test	20.0% / 11.1%	18.6% / 18.6%

Table 3.2.: Split hygiene of the two benchmarks, computed from the raw data. WDC Products is fully entity-disjoint; DBLP-Scholar is not, and additionally contains 186 test pairs whose serialized text is identical to a training pair.

WDC Products is clean. No entity from the test or validation split appears in training, no pair is shared between splits by ID, and no pair is shared by text either. Test performance on WDC Products therefore measures generalization to fully unseen products.

DBLP-Scholar is not entity-disjoint. 93.0% of the DBLP-side test entities and 63.5% of the Scholar-side test entities also appear somewhere in the training split. This is a known property of the pair-level random splits in the Magellan benchmark collection, not an error in our processing, but it matters for interpretation: a model can succeed on much of the DBLP-Scholar test set by recognizing entities it has already seen, so the near-ceiling baseline F1 of about 0.956 partly reflects memorization.

DBLP-Scholar contains text-identical cross-split pairs. While no pair is shared between train and test at the level of record IDs, the entity tables contain duplicate records, that is, the same paper appearing under different IDs. As a result, 186 test pairs (3.2% of the test set) are *textually identical* to a training pair, along with 160 train–validation and 64 test–validation duplicates. We state this precisely as “0 ID-level shared pairs; 186 text-identical pairs”, because the ID-level statement alone would give a false impression of cleanliness: for a text-based matcher, a textually identical pair is the same input regardless of its ID.

These facts shape the thesis in two ways. First, they define the hygiene bar for our own pipelines: on WDC Products, augmentation must use training entities only, preserving full disjointness; on DBLP-Scholar, where the baseline itself is not disjoint, augmentation must at least not use any entity that appears *exclusively* in validation or test (Section 4.6). Second, they pre-register an interpretation caveat: improvements are hard to obtain on DBLP-Scholar not only because the task is easy for transformers, but because part of the test set is effectively already known to the model.

4. Methods: Three Strategies for Training Set Expansion

This chapter describes the student model and the three expansion pipelines. Throughout, we use the distillation terminology introduced in Section 2.6: the large language model that labels pairs is the *teacher*, and the RoBERTa matcher trained on the resulting data is the *student*. Section 4.1 states the design requirements the expanded training sets should meet. Section 4.2 defines the student. Sections 4.3 to 4.5 present the three pipelines, and Section 4.6 describes the entity-disjointness policy that all pipelines must obey, including the leakage incident that motivated its strict enforcement.

4.1. Design Requirements for the Expanded Training Sets

An expanded training set should not just be bigger; it should be bigger in the right way. Based on the properties that make entity matching training sets effective, the pipelines were designed toward the following targets:

- **Label ratio:** roughly 1:3 positive to negative pairs (25% match rate), close to the ratio of the original training sets and consistent with what DistillER found to work for ranking-based selection (Zeakis et al. 2026).
- **Corner cases:** 40–50% of the added pairs should be hard (positives with low surface similarity or negatives with high surface similarity), since these are the pairs that teach a matcher something beyond token overlap.
- **Diversity:** no single entity cluster, brand, or category should dominate the added pairs.
- **Decisive attribute coverage** (WDC Products): the added pairs should cover variation in color, memory/storage, size, bundle contents, edition, and model number, the attributes that decide hard product cases.
- **Low label noise:** the teacher’s prompt must be designed carefully, and teacher labels must be checked against ground truth where possible (Section 5.6).
- **Entity-disjointness:** the added pairs may only use entities from the training split (Section 4.6).

Whether these targets were met is itself an empirical question, answered by the second evaluation layer in Section 6.4. Anticipating one result: the corner-case target turned

out to be structurally out of reach once entity-disjointness is enforced, and understanding why is one of the main findings of the thesis.

4.2. The Student Model

The student is a `roberta`-base model (Liu et al. 2019) with a two-class classification head, fine-tuned on serialized record pairs: the Ditto architecture (Li et al. 2020) reimplemented with current Hugging Face APIs (Wolf et al. 2020). A pair is encoded as `[CLS] left [SEP] right [SEP]`, truncated to 256 tokens, and classified as match or non-match. The original Ditto codebase targets Python 3.7 and outdated library versions; we use it as an architectural reference only and train with the standard `Trainer` (AdamW (Loshchilov and Hutter 2019), weight decay 0.01, warmup of 100 steps). Full hyperparameters are listed in Chapter 5 and Appendix B.

One adaptation matters for WDC Products. With a 20% positive rate and a small training set, the default cross-entropy objective produced a student biased against the minority match class. We therefore train a class-weighted variant for WDC Products, weighting each class inversely proportional to its frequency ($w_c = n/(2n_c)$). This raised baseline F1 from 0.608 to 0.641 in the initial single-seed configuration and is used consistently for all WDC Products runs (marked with the suffix `_cw` in result tables). DBLP-Scholar, with more data and a milder imbalance, did not benefit from class weighting, so its runs use the plain objective.

The student, its hyperparameters, its validation-based model selection, and its evaluation are held *fixed* across all experiments. The only thing that varies between runs is the training file. This is the design principle of the whole thesis: every comparison is a comparison between training sets, not between models.

4.3. Strategy 1: String-Based Augmentation

The first strategy is the classic one, ported from Ditto’s augmentation module (Li et al. 2020). It operates directly on the serialized strings and creates modified copies of existing training pairs. Three operators are used, each of which only ever touches value tokens; the `COL/VAL` structure markers and attribute names are never modified, so every augmented string remains a valid serialization:

- **Span deletion (`del`):** delete a random span of 1–2 consecutive value tokens.
- **Span shuffle (`swap`):** shuffle a random span of 2–4 consecutive value tokens in place.
- **Attribute drop (`drop_col`):** remove one attribute block entirely, preferring short attributes (at most 8 tokens) so that little information is lost.

Following Ditto’s RandAugment-style `all` mode, each augmented record receives three operations drawn at random from this set. For every training pair, one augmented copy

is created by independently augmenting the left and the right record while keeping the label. The output file contains the original pairs followed by the augmented copies, doubling the training set: 5,000 pairs for WDC Products and 34,446 for DBLP-Scholar.

The underlying assumption is label preservation: dropping a couple of tokens or re-ordering a span should not turn a match into a non-match. This is usually true, but not always: deleting exactly the token `500GB` from one side of a hard negative can remove the only evidence of the non-match. We accept this small noise rate, as Ditto does, and note that the risk is highest precisely on corner cases. String augmentation adds no new entities, no new pairings, and no new labels; it is the control condition that any more expensive strategy must beat.

4.4. Strategy 2: LLM-Based Augmentation with Active Learning

The second strategy creates truly *new* pairs (pairings of training entities that the benchmark does not contain) and labels them with the LLM teacher. The pipeline follows the active-learning recipe of [Steiner and Bizer \(2026\)](#) in four steps: embedding-based blocking builds a candidate pool, a seeding step labels an initial set of pairs, an active-learning loop iteratively labels the most uncertain candidates, and the labeled pairs are appended to the original training set. The teacher for all labeling steps is Claude (`claude-sonnet-4-6`), prompted per pair with the templates shown in [Appendix A](#).

4.4.1. Step 1: Embedding-Based Blocking

All entity records are embedded with the sentence-transformer `all-MiniLM-L6-v2` ([Reimers and Gurevych 2019](#); [Wang et al. 2020](#)), applied to the same serialized text the student sees. For each left entity, the $k = 20$ nearest right-side neighbors by cosine similarity form the raw candidate set. This produced 57,920 raw candidate pairs for WDC Products and 52,320 for DBLP-Scholar.

4.4.2. Step 2: Filtering the Pool

Two filters are applied to the raw candidates. First, every pair that already appears in the benchmark (in train, validation, or test) is removed, because the goal is to add new pairs, not to duplicate or relabel existing ones (3,171 known pairs removed for WDC Products, 10,925 for DBLP-Scholar). Second, every pair that touches an entity outside the training split is removed (47,236 pairs for WDC Products, 22,554 for DBLP-Scholar); [Section 4.6](#) explains why this filter is essential and what happened without it. The final pools contain **4,722** candidate pairs for WDC Products and **18,841** for DBLP-Scholar, each carrying its embedding cosine similarity as metadata.

[Figure 4.1](#) shows the similarity profile of the resulting pools: by construction of the k -nearest-neighbor blocking, both are concentrated at high cosine similarity (mean 0.83 on WDC Products, 0.75 on DBLP-Scholar); these are exactly the look-alike pairs a matcher finds hard.

4. Methods: Three Strategies for Training Set Expansion

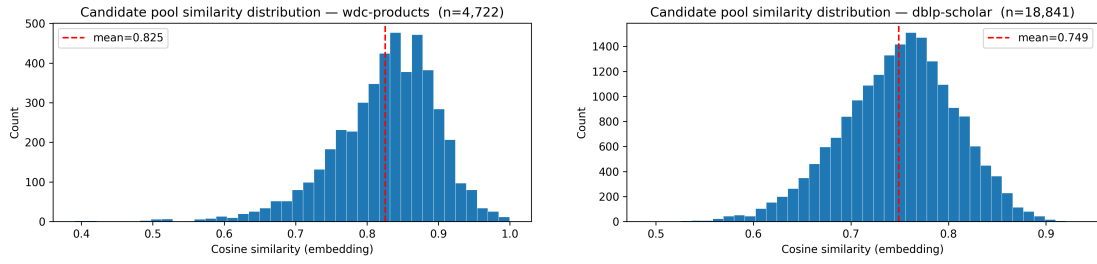


Figure 4.1.: Embedding cosine similarity of the filtered candidate pools (left: WDC Products, 4,722 pairs; right: DBLP-Scholar, 18,841 pairs). Nearest-neighbor blocking concentrates both pools at high similarity: plausible-looking pairs whose labels are not obvious from the surface.

The composition of these pools is worth pausing on, because it determines everything downstream. Using the cluster ground truth of WDC Products (which the pipeline itself never accesses), we can count how many candidates are true matches: only 266 of 4,722 (5.6%). The corresponding number for DBLP-Scholar, as the seeding step will reveal, is effectively zero. The reason is structural. On a closed benchmark, the true matches among the benchmark’s own entities are exactly what the benchmark was built to enumerate, so they are already in the labeled splits and are removed by the known-pair filter. What survives into the pool is the complement: pairs that look similar (they passed blocking) but were not included as matches, which is to say, mostly hard negatives. We return to this repeatedly; it is the single most consequential fact in the thesis.

4.4.3. Step 3: Seed Labeling

The seeding step gives the active-learning committee something to start from. Left entities are visited in order of their most similar candidate (most promising first). For each entity, its candidates are labeled by the teacher in decreasing similarity order until at least one positive and two negatives have been found, up to a cap of 8 candidates per entity; in addition, up to 2 candidates from the *bottom* of the entity’s similarity ranking are labeled, so that the committee also sees clearly dissimilar pairs. The process stops at 100 labeled seed pairs per dataset.

The teacher prompt (Appendix A) instructs the model to decide whether the two records refer to the exact same product (“same model, variant, and specs, not just the same product line”) and, for WDC Products, explicitly lists the decisive attributes (color, storage capacity, size, bundle contents, edition, model number) that the design requirements call for. The teacher must answer in strict JSON with a label and a one-sentence reasoning; malformed responses are retried up to three times. The reasoning sentences are stored with every labeled pair and were used for spot-checking label quality during development.

On DBLP-Scholar, seeding already delivered the central bad news: among all its

labeled pairs, the teacher found *zero* positives. The pool simply does not contain discoverable new matches.

4.4.4. Step 4: Active Learning Loop

The main labeling budget is spent by a committee-based active-learning loop, run for 20 iterations of 100 pairs each (2,000 teacher labels per dataset). In each iteration:

1. A committee of four classifiers (random forest, XGBoost, gradient boosting, and logistic regression (Breiman 2001; Chen and Guestrin 2016; Friedman 2001)) is trained on all labeled data so far: the original training pairs, the seed labels, and all active-learning labels from previous iterations.
2. Each classifier operates on six similarity features per pair: token-level Jaccard, character-bigram Jaccard, Jaro–Winkler similarity, Levenshtein ratio, token-set ratio, and the embedding cosine from blocking. Features are computed on the serialized text with the COL/VAL markers stripped.
3. Every remaining pool pair is scored by all four classifiers, and the *variance* of the four match probabilities is taken as the uncertainty signal. The 100 highest-variance pairs, those the committee disagrees on most, are sent to the teacher for labeling and added to the labeled set.

After 20 iterations, the seed and active-learning labels together contribute 2,100 new pairs per dataset. Appended to the original training sets, this yields `train_aug_llm` files with 4,600 pairs for WDC Products and 19,308 for DBLP-Scholar.

Two deviations from the recipe of Steiner and Bizer (2026) should be stated openly. Their Step 4 adds a final 20% of randomly sampled pool pairs to counteract the selection bias of uncertainty sampling, and their Step 5 uses a separate LLM-labeled validation set for model selection. We implemented neither: the labeled set consists of the seed and uncertainty-selected pairs only, and student model selection uses the benchmark’s own validation split (as it does for every other strategy, keeping comparisons clean). The random top-up would have mattered more with a large pool; our WDC Products pool of 4,722 pairs is small relative to the 2,100 labels spent on it (44%), so the labeled set covers the pool densely anyway.

The label composition of the result reflects the pool composition described above. On WDC Products, 217 of the 2,100 added pairs (10.3%) received a positive teacher label, pulling the positive rate of the combined training set down from 20.0% to 15.6%. On DBLP-Scholar, *none* of the 2,085 added pairs (after deduplication) is positive: the expanded training set gains 2,085 teacher-labeled hard negatives and not a single new match. Figure 4.2 makes this visible per iteration: the WDC Products positive rate hovers around 10% and trends downward as the pool’s few matches are used up, while the DBLP-Scholar curve is a flat line at zero across all 20 iterations. The active-learning machinery worked exactly as intended (it found the disagreement region of the pool), but the pool had no positives to find.

4. Methods: Three Strategies for Training Set Expansion

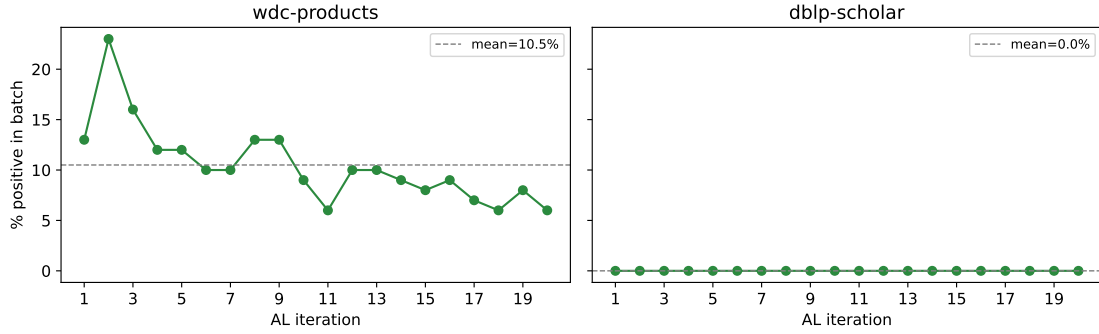


Figure 4.2.: Share of positive teacher labels per active-learning iteration (100 pairs each). On WDC Products (left) the rate averages about 10% and declines over the iterations; on DBLP-Scholar (right) not a single positive is found among 2,000 labeled pairs. This is the positive-scarcity result of Section 4.4.2 in one picture.

4.5. Strategy 3: Web Retrieval Augmentation

The third strategy attacks the limitation that the first two cannot escape: everything so far recombines or perturbs records the benchmark already contains. Web retrieval brings in *new* records from outside. The idea is simple: for a product in the training set, other shops publish offers for the same product with different wording; for a paper, other bibliographic sites publish variant records. Retrieving such documents and pairing them with the original entity yields pairs whose positive side really adds information. The pipeline has four steps.

4.5.1. Step 1: Entity Sampling

Labeling budget is limited, so we do not retrieve for every training entity. Per dataset, 150 entities are sampled from each side (300 total), stratified to protect diversity: entities are grouped into buckets by **brand** (WDC Products) or **venue** (DBLP-Scholar), and no bucket may contribute more than 15% of a side’s sample. Sampling is restricted to training entities (Section 4.6). For each sampled entity, a search query is built from its identifying attributes (brand and title for products, title and authors for papers) and truncated to 400 characters.

4.5.2. Step 2: Web Search

Each query is sent to the Tavily search API,¹ requesting up to 5 results with basic search depth. Each result consists of a page title, URL, and a content snippet. The top 3 results per entity are passed to the next step; snippets are truncated to 800 characters.

¹<https://tavily.com>

4.5.3. Step 3: Extraction and Labeling

Raw web snippets are not records, so the teacher performs three subtasks in a single prompt (Appendix A): (1) decide whether the snippet describes a *specific* product or paper at all (category pages, blog posts, and search-result pages are marked irrelevant and discarded); (2) if relevant, extract a structured record with exactly the benchmark’s attribute schema, using empty strings for attributes the snippet does not mention; (3) decide whether the extracted record matches the original entity, using the same match definition and decisive-attribute instructions as in Strategy 2. The output is a pair (original record, extracted web record) with a teacher label.

This step is where the strategy’s characteristic bias enters. Because the query is built from the entity’s own brand and title, the search engine mostly returns pages about *that product*, which is the point, but it means the harvested pairs are predominantly positives. The relevant slice for WDC Products came out at 81% positive. This skew is inherent to retrieval by identity (searching for a product finds that product), and Chapter 7 describes a follow-up experiment that tried to engineer the complementary hard negatives deliberately.

4.5.4. Step 4: Finalization

The relevant labeled pairs are serialized into the benchmark schema and appended to the original training set. For WDC Products this added 803 pairs (yielding 3,303 total, 34.8% positive); for DBLP-Scholar, 742 pairs (17,965 total, 21.4% positive). Before training, a random sample of the labeled pairs was manually reviewed for label noise; the labels were found to be plausible, with the caveat that web snippets are sometimes too thin to decide variant-level questions with certainty.

Web retrieval is the only strategy of the three that adds new positive *entities*, new descriptions of training products from other shops, and is therefore the only one that addresses the positive-scarcity problem of Strategy 2 head-on. Its weakness is textual: extracted web records are written differently from the benchmark’s records (marketing copy, truncated snippets, missing attributes), so the added pairs are of type (benchmark record, web record) while the test set consists purely of (benchmark record, benchmark record) pairs. The consequences of this distribution mismatch surface in Chapters 6 and 7.

4.6. Ensuring Entity-Disjoint Augmentation

All three pipelines share one hygiene rule: **augmentation may only use entities from the training split**. String augmentation satisfies it trivially, since it only rewrites existing training pairs. The LLM and web pipelines had to be made to satisfy it, and were not in their first version.

The first version of the blocking step (Section 4.4.1) built its candidate pool over the *full* entity tables and excluded only known *pairs*, on the reasoning that no benchmark pair would then be duplicated. That reasoning misses the point of entity-disjointness.

4. Methods: Three Strategies for Training Set Expansion

The WDC Products entity tables contain about 2,900 entities per side, of which only about 1,000 appear in training; the rest are validation and test entities. Blocking over all of them meant that 59% of the added LLM pairs on WDC Products (and 78% on DBLP-Scholar) contained at least one held-out entity, with teacher labels attached. No exact test pair was leaked, but the student saw the *text* of test entities during training, on a benchmark whose test set is specifically constructed from unseen entities. The web pipeline had the analogous flaw in its entity sampling. An independent methodology audit caught the problem after the first full round of results.

The fix has three parts. First, a single module now serves as the authoritative source of entity-to-split membership, handling a subtlety that makes an ad-hoc fix dangerous: WDC Products uses one shared ID space (the same entity may appear on either side of a pair, so an excluded entity must be filtered from both sides), while DBLP-Scholar uses two separate per-side ID spaces whose numeric IDs overlap (so filtering must be side-specific, or a DBLP ID would wrongly exclude an unrelated Scholar record). Second, both pipelines filter their pools through this module: a candidate pair survives only if both entities belong to the training split. Third, a verification script recounts, for every augmented training file, how many pairs touch a held-out entity, i.e. an entity appearing in validation or test but *not* in training. The pass criterion is zero, and the final regenerated training sets meet it on both datasets. On WDC Products, whose benchmark splits are entity-disjoint, the augmented sets are therefore fully test-disjoint. On DBLP-Scholar, where most test entities also appear in training (Section 3.4), the policy yields augmentation that is exactly as disjoint as the baseline itself: it introduces no entity the original training set did not already use, which is the strongest guarantee possible on a non-disjoint benchmark.

The cost of honesty was substantial and is a finding in itself. All LLM-pipeline and web-pipeline data was regenerated on the clean pools, all affected models retrained, and the results re-evaluated. The clean WDC Products pool lost most of its positives (the leaked version had a healthy $\sim 25\%$ positive rate, supplied, as it turned out, by held-out entities pairing with their training-set counterparts), and the clean DBLP-Scholar pool lost all of them. Every number reported in this thesis comes from the clean, regenerated data; where the contrast with the leaked version is instructive, it is reported explicitly and labeled as such.

5. Experimental Setup

This chapter fixes everything that is held constant across experiments: the training configuration of the student (Section 5.1), the metrics (Section 5.2), the multi-seed protocol (Section 5.3), the statistical tests (Section 5.4), the three-layer evaluation scheme (Section 5.5), and the teacher-quality measurement (Section 5.6).

5.1. Training Configuration

Every student is a `roberta-base` model fine-tuned for up to 10 epochs with batch size 8 and gradient accumulation over 4 steps (effective batch size 32), maximum sequence length 256, AdamW with weight decay 0.01, and 100 warmup steps. After each epoch the model is evaluated on the benchmark’s validation split; the checkpoint with the best validation F1 is kept, and training stops early if validation F1 does not improve for 3 consecutive epochs. WDC Products students use the class-weighted loss of Section 4.2; DBLP-Scholar students use plain cross-entropy.

The learning rate needs a word of explanation. The initial single-seed experiments used $5 \cdot 10^{-5}$. During the multi-seed reruns, this rate occasionally caused training collapse on the imbalanced augmented sets (the student degenerating to the majority class), so the final protocol uses $2 \cdot 10^{-5}$ for all runs. All headline results in this thesis come from the $2 \cdot 10^{-5}$, three-seed protocol. Where earlier single-seed numbers at $5 \cdot 10^{-5}$ are mentioned for contrast, the learning-rate difference is acknowledged as a confound: the “single-seed vs. multi-seed” comparisons in this thesis bundle a seed change with a learning-rate change, and we do not attribute the differences to seed variance alone.

At evaluation time, the trained student predicts on the test split with the default argmax decision rule (equivalent to a 0.5 threshold on the match probability); Section 6.1.1 examines threshold calibration separately. Per-pair predictions and match scores are saved for every run, which enables the paired tests and error analyses described below, and allows every reported number to be recomputed from the stored predictions.

5.2. Metrics

The headline metric is F1 on the positive (match) class, with precision and recall reported alongside (Section 2.1). We do not use accuracy as a headline metric: with test positive rates of 11–19%, accuracy is dominated by the majority class, and a matcher that rejects everything would already reach 0.89 accuracy on WDC Products. F1 is also what both

5. Experimental Setup

comparison systems report (Zeakis et al. 2026; Steiner and Bizer 2026), which keeps Chapter 8 on one scale.

5.3. Multi-Seed Protocol

Fine-tuning a 125M-parameter model on a few thousand pairs is noisy: different random seeds change weight initialization of the classification head and data ordering, and with them the final F1. The first round of experiments in this thesis used a single seed, and several of its conclusions (including a statistically significant improvement of one strategy and a significant degradation of another) turned out to be artifacts of that seed (Section 6.3). The final protocol therefore trains every configuration three times with seeds 13, 42, and 87, and reports F1 as mean $\pm$ standard deviation across seeds. The standard deviation is the sample estimator (dividing by $n-1$), matching how Steiner and Bizer (2026) and DistillER report variation across runs; with $n=3$ the estimate is coarse, but it reliably reveals when a delta between two configurations is smaller than the run-to-run noise of either.

Three seeds are a compromise dictated by compute (each configuration is a full fine-tuning run), and they set the resolution limit of the study: on WDC Products the observed seed noise is ± 0.015 – 0.020 F1, so mean differences below roughly 0.02 cannot be distinguished from noise at this sample size. Claims in this thesis are phrased accordingly: “no detectable effect at $n=3$ ” rather than “no effect”.

5.4. Statistical Significance Testing

Whenever two students are compared, both are evaluated on the *same* test pairs, so the comparison is paired. Unpaired tests (such as the two-proportion z -test on accuracy) throw away the pairing and are the wrong tool; an earlier version of our analysis used one, and switching to the correct paired test changed two conclusions on DBLP-Scholar. Following Dietterich (1998), model-versus-model comparisons use **McNemar’s test** (McNemar 1947) on the discordant pairs (test items that exactly one of the two models classifies correctly) with the exact two-sided binomial p -value.

McNemar’s test operates on binary correctness, not on F1. To quantify uncertainty on F1 itself we additionally report a **paired bootstrap** (Efron and Tibshirani 1994): test pairs are resampled with replacement 10,000 times, the same resampled indices are applied to both models to preserve pairing, and a percentile confidence interval is computed for each model’s F1 and for the F1 difference. As an effect size we report **Cohen’s h** on accuracy (Cohen 1988), so that statistical significance is always accompanied by a measure of practical significance: with test sets of 4,500–5,742 pairs, tiny effects can be significant.

Because the multi-seed protocol produces three prediction sets per configuration, the paired tests are run on *seed-averaged predictions*: the per-pair match scores of the three seeds are averaged and thresholded at 0.5, and McNemar’s test compares these ensemble predictions. To be precise about what this tests: it compares the score-averaged

5. Experimental Setup

ensembles of the two configurations, not the distribution of per-seed results; with $n=3$ seeds, a test across seeds would have essentially no power, and the ensemble comparison is the practical alternative. Finally, the thesis runs roughly seventeen such comparisons at $\alpha = 0.05$; under a Bonferroni correction (Dunn 1961) the threshold becomes ≈ 0.003 , and we mark which of the significant results survive it. Only two do: one degradation and one improvement.

5.5. Three-Layer Evaluation Scheme

Downstream F1 alone cannot explain *why* a strategy helps or fails, so every strategy is evaluated on three layers:

1. **Standard metrics:** precision, recall, F1 of the student on the test split, under the protocol above.
2. **Training set characteristics:** properties of the expanded training file itself, namely size, positive rate, corner-case proportion, coverage of decisive attributes (WDC Products), and pairwise overlap between the pairs added by different strategies. Corner cases are defined via token-level Jaccard similarity between the two records (computed on value tokens, with serialization markers stripped): a *hard positive* is a match with similarity below 0.3, a *hard negative* a non-match with similarity above 0.4. The asymmetric thresholds reflect the asymmetry of the two error types: matches routinely share many tokens, so a match only becomes hard when overlap is very low, whereas a non-match becomes confusable already at moderate overlap. The middle band (0.3–0.4) is deliberately left neutral. These thresholds are heuristic; they are used consistently across all strategies and datasets, so comparisons between strategies are on equal footing even if the absolute corner-case percentages shift with the threshold choice.
3. **Qualitative error analysis:** misclassified test pairs of each student are grouped into four classes: highly similar non-matches (false positives with similarity > 0.5), dissimilar matches (false negatives with similarity < 0.2), noisy or incomplete records (one side shorter than six value tokens), and the remaining ambiguous variant cases. Counts and representative examples are reported per class. Two complementary views are added: the *overlap* of the error sets of two students (Jaccard similarity over misclassified test-pair indices, answering whether an augmented student fails on the same pairs as the baseline), and a *hard-subset evaluation* that reports F1 restricted to the corner-case pairs of the test set.

The second layer directly checks the design requirements of Section 4.1; the third shows where each student actually fails and whether the failure profile changes with the training set.

5.6. Measuring Teacher Label Quality

The design requirements demand low teacher label noise, and “low” should be a number, not an impression. For WDC Products this can be measured exactly: the benchmark’s cluster ground truth determines the true label of *any* pair of its entities, including the new pairs our pipelines created, which the benchmark never labeled explicitly. We therefore score every teacher-labeled pair from the LLM pipeline (100 seed + 2,000 active-learning labels) against the cluster ground truth and report the teacher’s accuracy, precision, recall, and F1, overall and per pipeline stage. The pipeline itself never sees the cluster IDs; they are used only for this audit. For DBLP-Scholar no per-entity ground truth exists for new pairs, so teacher quality there rests on the manual spot checks mentioned in Section [4.5.4](#).

6. Results

This chapter reports the main experimental results. Section 6.1 establishes the baselines and examines the WDC Products threshold question. Section 6.2 presents the downstream results of the three expansion strategies (Layer 1). Section 6.3 documents how the multi-seed protocol overturned earlier single-seed conclusions. Sections 6.4 and 6.5 analyze the expanded training sets and the teacher’s label quality, and Section 6.6 presents the error analysis (Layer 3). Section 6.7 condenses the chapter into its main findings.

6.1. Baselines

Table 6.1 shows the baseline students trained on the unmodified training sets. On DBLP-Scholar, **roberta-base** reaches an F1 of 0.9559 ± 0.0019 , the expected near-ceiling result for a transformer on this benchmark, with the memorization caveat from Section 3.4; it also lands right on the 0.956 that Ditto reports for this dataset (Li et al. 2020) (Section 2.2). On WDC Products the class-weighted baseline reaches 0.659 ± 0.015 ; the run-to-run spread is an order of magnitude larger than on DBLP-Scholar (per-seed F1: 0.677, 0.649, 0.653), which quantifies how unstable small-data fine-tuning is on this benchmark and why single-seed comparisons on it are unreliable. Figure 6.1 shows the error structure of a representative WDC Products baseline: false positives outnumber false negatives by more than two to one (297 versus 124), the over-prediction pattern that the prior-shift analysis below explains.

Dataset	Run	Precision	Recall	F1
WDC Products	baseline (class-weighted)	0.618 ± 0.056	0.715 ± 0.070	0.659 ± 0.015
DBLP-Scholar	baseline	0.952 ± 0.004	0.960 ± 0.001	0.9559 ± 0.0019

Table 6.1.: Baseline students (3-seed mean $\pm$ sample standard deviation, seeds 13/42/87).

6. Results

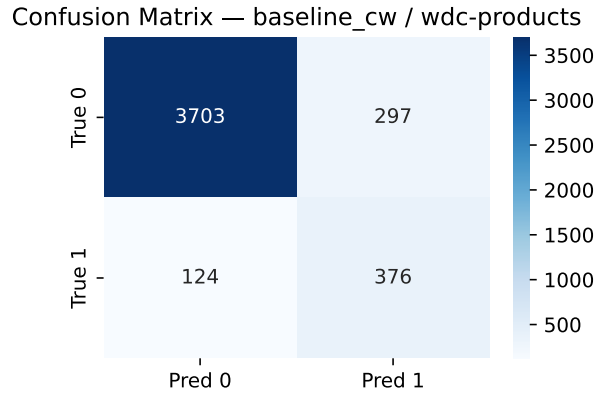


Figure 6.1.: Confusion matrix of the class-weighted WDC Products baseline (initial single-seed run; test split, $n = 4,500$). The model over-predicts the match class (297 false positives against 124 false negatives), consistent with a decision rule calibrated to the 20% training prior and applied to an 11% test prior.

6.1.1. The WDC Products Ceiling and the Prior Shift

Our WDC Products baseline sits below the 0.70–0.80 range that Ditto reports on its product benchmarks (Li et al. 2020). Part of the explanation is the deliberately hard 80%-corner-case configuration and the small training size, but a second, less obvious part is the prior shift documented in Section 3.4: the student is trained, validated, and early-stopped against a 20% positive rate, then tested against 11.1%. A model calibrated to the training prior will over-predict matches on the test set at the default threshold.

To quantify how much of the gap the decision threshold could recover, we calibrated the threshold of the class-weighted baseline in four ways (single-seed analysis; Appendix C.1 gives the full table). The default argmax rule yields F1 0.641. Tuning the threshold for maximum F1 on the *validation* split makes things slightly worse (0.632 at threshold 0.444): validation mirrors the training prior, not the test prior, so it pulls the threshold in the wrong direction. Matching the threshold to the known test prior costs even more (0.565). The oracle threshold, tuned directly on the test set (an upper bound no fair method can reach), is far above the default (0.843) and yields 0.659, a gain of just +0.018. The conclusion is twofold: the model indeed over-predicts positives at the default cut, confirming the prior-shift diagnosis, but no legitimately selectable threshold recovers more than a fraction of the gap. The WDC Products ceiling of roughly 0.64–0.68 in our setting is thus substantially a property of the benchmark configuration, and we evaluate all strategies at the common default threshold.

6.2. Downstream Results of the Three Strategies

Tables 6.2 and 6.3 present the headline results: the three expansion strategies against the baseline, per dataset, under the full protocol of Chapter 5 (three seeds, McNemar on seed-averaged predictions, paired bootstrap on F1). Figure 6.2 shows the same picture graphically, together with the second-iteration configurations that Chapter 7 discusses; the visual message is that every error bar overlaps the baseline’s on WDC Products, while on DBLP-Scholar the differences are barely visible at all.

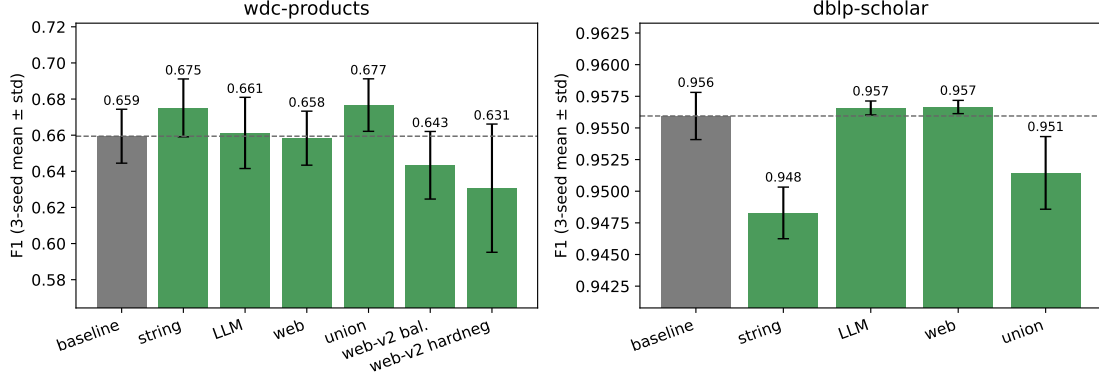


Figure 6.2.: Test F1 of all evaluated configurations (3-seed mean $\pm$ sample std). The dashed line marks the baseline mean. Left: WDC Products (class-weighted runs), including the union and web-v2 variants of Chapter 7. Right: DBLP-Scholar. No configuration separates from the baseline by more than the seed noise except the web-v2 variants, which are significantly worse.

Run	Precision	Recall	F1	Δ F1	McNemar p
baseline	0.618 ± 0.056	0.715 ± 0.070	0.659 ± 0.015	–	–
string_aug	0.618 ± 0.051	0.751 ± 0.068	0.675 ± 0.016	+0.016	0.244 (n.s.)
llm_aug	0.587 ± 0.031	0.759 ± 0.050	0.661 ± 0.020	+0.002	0.356 (n.s.)
web_aug	0.589 ± 0.043	0.750 ± 0.035	0.658 ± 0.015	−0.001	0.396 (n.s.)

Table 6.2.: WDC Products test results (3-seed mean $\pm$ std; all runs class-weighted). No strategy differs significantly from the baseline.

On WDC Products, no strategy beats the baseline. String augmentation has the best mean (+0.016 F1) and wins on two of three seeds, but it loses on the third, its delta sits inside the ± 0.015 – 0.020 seed-noise band, and McNemar’s test on the seed-averaged predictions is far from significance ($p = 0.244$). The LLM and web strategies land within ± 0.002 of the baseline: statistical ties. Recall shifts upward under every augmentation ($0.715 \rightarrow 0.750$ – 0.759) at the cost of precision, consistent with all three strategies exposing the student to more positive-region variation without teaching it the decisive distinctions that would protect precision.

6. Results

Run	Precision	Recall	F1	$\Delta F1$	McNemar p
baseline	0.952 ± 0.004	0.960 ± 0.001	0.9559 ± 0.0019	–	–
string_aug	0.927 ± 0.007	0.971 ± 0.005	0.9483 ± 0.0020	−0.008	0.0079 (worse)
llm_aug	0.952 ± 0.006	0.961 ± 0.006	0.9566 ± 0.0006	+0.001	0.728 (n.s.)
web_aug	0.948 ± 0.006	0.966 ± 0.007	0.9567 ± 0.0005	+0.001	1.000 (n.s.)

Table 6.3.: DBLP-Scholar test results (3-seed mean $\pm$ std). Only string augmentation differs significantly from the baseline: it is worse; the effect is nominal under Bonferroni correction.

On DBLP-Scholar, the baseline is unbeaten, and one strategy reliably hurts. String augmentation loses on all three seeds (-0.008 F1, $p = 0.0079$); the effect is tiny (Cohen’s $h \approx 0.03$) and only nominal under Bonferroni correction, but its direction is consistent. The mechanism is visible in the precision–recall split: doubling the training set with perturbed copies pushes recall up ($0.960 \rightarrow 0.971$) but costs more precision ($0.952 \rightarrow 0.927$), a bad trade on a dataset where the baseline’s errors are already concentrated in a thin band of truly ambiguous pairs (Section 6.6). Perturbation noise near the ceiling erases signal rather than adding robustness: some perturbed copies of hard negatives lose exactly the tokens that made them negatives (the risk anticipated in Section 4.3). The LLM and web strategies tie the baseline ($p = 0.73$ and 1.0): 2,085 clean teacher-labeled hard negatives (LLM) and 742 web pairs changed nothing measurable.

Answering RQ1–RQ3 directly: at the full labeling budget, none of the three strategies improves the student at a level distinguishable from seed noise, on either dataset. The remainder of the thesis is about *why* (the answer differs per strategy) and about where the picture changes: Section 7.4 shows that the LLM strategy *does* help significantly once the base training set is made scarce.

6.3. What Multi-Seed Evaluation Changed

The first complete round of experiments used a single seed (42) at learning rate $5 \cdot 10^{-5}$, and its conclusions differed sharply from Table 6.2: WDC string augmentation appeared to deliver a large, highly significant gain ($+0.035$, $p < 0.001$), WDC LLM augmentation appeared significantly *harmful* (-0.027 , $p = 0.0004$), and DBLP-Scholar LLM augmentation also appeared significantly worse. Under the three-seed protocol, all three conclusions dissolved: the string gain shrank to a non-significant $+0.016$, and both apparent LLM degradations turned out to be seed-42 outliers: over three seeds, LLM augmentation *ties* the baseline on both datasets.

Two lessons follow. First, the WDC Products baseline itself varies by 0.028 F1 across our three seeds, so any single-seed delta below roughly ± 0.03 on this benchmark is uninterpretable, and most reported augmentation effects in the literature are of exactly that size. Second, as flagged in Section 5.1, our protocol change bundled the seed change with a learning-rate change, so we avoid attributing the flips to seed variance alone; the honest statement is that the initial conclusions were not robust to a reasonable change

of training conditions, which is precisely what robustness protocols exist to detect.

6.4. Training Set Characteristics (Layer 2)

Dataset	Training set	N	% Pos.	Hard pos.	Hard neg.	% Corner
WDC	baseline	2,500	20.0	435	65	20.0
	string_aug	5,000	20.0	882	123	20.1
	llm_aug	4,600	15.6	595	211	17.5
	web_aug	3,303	34.8	980	66	31.7
DBLP	baseline	17,223	18.6	200	344	3.2
	string_aug	34,446	18.6	935	567	4.4
	llm_aug	19,308	16.6	200	385	3.0
	web_aug	17,965	21.4	337	351	3.8

Table 6.4.: Layer 2: composition of the expanded training sets. Corner cases: hard positive = match with token-Jaccard < 0.3 ; hard negative = non-match with token-Jaccard > 0.4 .

Table 6.4 checks the expanded training sets against the design requirements of Section 4.1, and Figure 6.3 shows the underlying similarity distributions per training set, split by label, with the corner-case thresholds marked. The headline is that the two central targets were missed everywhere.

The best corner-case proportion achieved on WDC Products is 31.7% (web augmentation) against a target of 40–50%; on DBLP-Scholar the best is 4.4%, and no strategy even approaches the target. The positive-ratio picture is similarly mixed: LLM augmentation *lowered* the positive rate on both datasets (to 15.6% and 16.6%), and web augmentation overshot on WDC Products (34.8%).

Three per-strategy observations explain these numbers.

LLM augmentation is positive-starved by construction. As established in Section 4.4.2, the clean candidate pool contains almost no true matches (5.6% on WDC Products, none discoverable on DBLP-Scholar), so the added slice is 10.3% positive on WDC Products and 0% positive on DBLP-Scholar. It does deliver one design goal: it is the only strategy that meaningfully increases *hard negatives* on WDC Products (65 $\rightarrow$ 211), exactly what uncertainty sampling should find in a negative-dominated pool. For contrast, the leaked pre-fix version of this training set had a comfortable $\sim 25\%$ positive rate and 27.5% corner cases: numbers that met the targets and were entirely an artifact of held-out entities supplying the missing positives. The targets, in other words, were calibrated on what a leaky pipeline can achieve, and become structurally unreachable once the pipeline is honest.

Web augmentation has the opposite skew. Its added slice alone is 81% positive (68% corner cases) on WDC Products, diluted to 34.8% only by the base training set, for the retrieval-by-identity reason given in Section 4.5.3. It is also the only strategy whose

6. Results

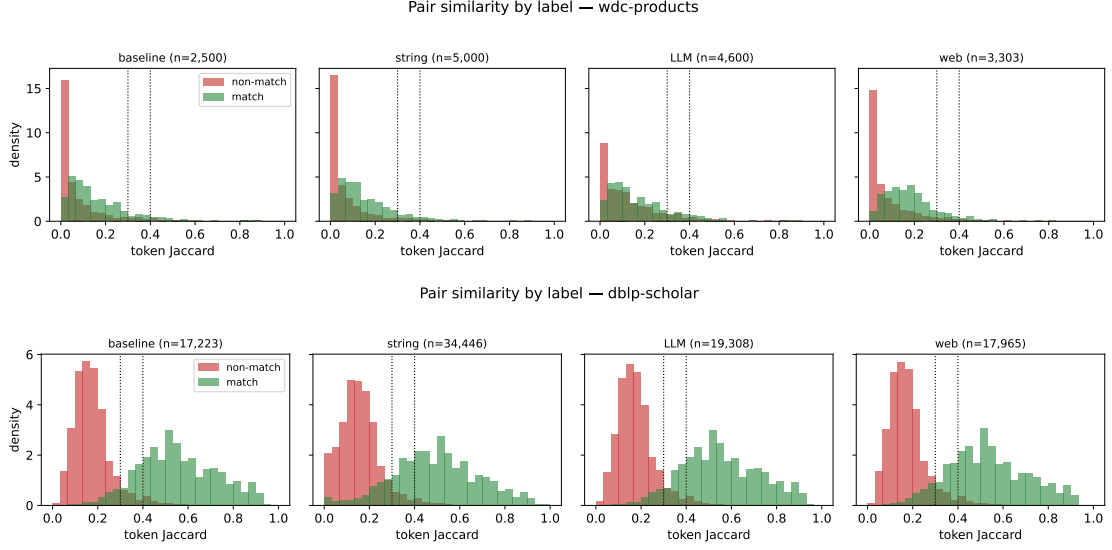


Figure 6.3.: Pair-similarity distributions (token-level Jaccard) of the four training sets, split by label; top row WDC Products, bottom row DBLP-Scholar. Dotted lines mark the corner-case thresholds (hard positive < 0.3 , hard negative > 0.4). On WDC Products the match and non-match distributions overlap broadly and the corner-case band is populated; on DBLP-Scholar they are almost perfectly separated, which is the visual form of the dataset’s 3–4% corner-case share.

hard-positive count rises sharply (980), since web descriptions of a training product often share few tokens with the benchmark’s description of it.

DBLP-Scholar structurally lacks corner cases. Every strategy lands at 3–4% corner cases on DBLP-Scholar, against 20–32% on WDC Products. The bottom row of Figure 6.3 shows why: the match and non-match similarity distributions barely overlap. Paper records are either nearly identical (same paper) or clearly different (different papers); the borderline band that makes products hard barely exists; title duplicates across truly different papers are rare. This single number goes a long way toward explaining both the 0.956 ceiling and the failure of every augmentation to move it.

Finally, the pairwise overlap between the added slices of the three strategies is exactly **zero** on both datasets: no pair is produced by more than one strategy. String augmentation rewrites existing pairs, LLM augmentation pairs training entities in new combinations, and web augmentation imports external records, so they explore disjoint regions of pair space by construction. The three strategies are perfectly complementary in what they add, which motivates the union experiment of Section 7.3: complementary additions can be combined without redundancy.

Attribute coverage (the remaining requirement) is stable: all strategies stay within a few points of the baseline’s coverage of color ($\sim 35\%$), memory/storage ($\sim 34\text{--}40\%$), size ($\sim 37\%$), bundle ($\sim 13\%$), edition ($\sim 20\%$), and model number ($\sim 30\text{--}35\%$) mentions on

WDC Products; the full table is in Appendix C.3. No strategy materially expands coverage of the underrepresented attributes; none was designed to target attributes directly.

6.5. Teacher Label Quality

Scoring all 2,100 teacher-labeled WDC Products pairs against the cluster ground truth (Section 5.6) yields the following picture: accuracy **0.971**, precision 0.866, recall 0.858, F1 **0.862**, i.e. an overall label-noise rate of 2.9% (29 false matches, 31 missed matches out of 2,100). Split by pipeline stage, the 100 similarity-ordered seed pairs were labeled with *zero* errors, while the 2,000 uncertainty-selected active-learning pairs carry 3.0% noise, which is not surprising, since uncertainty sampling deliberately routes the hardest pairs to the teacher.

Two conclusions follow. First, the teacher meets the low-noise design requirement: 97% agreement with ground truth on adversarially selected pairs is a good annotator by any human-annotation standard. Second, and more importantly for interpreting the thesis, teacher noise is *not* the bottleneck of LLM-based expansion. The teacher’s labeling F1 (0.862) far exceeds the student’s downstream F1 on this benchmark (~ 0.66); what the strategy lacked was not label quality but *positives to label* (Section 6.4). This cleanly separates our negative result from the failure mode DistillER centers on (noisy teachers), and the point is taken up again in Chapter 8.

6.6. Error Analysis (Layer 3)

The error analysis inspects the misclassified test pairs of each student under the classification scheme of Section 5.5. The counts reported in the text come from the initial single-seed runs (they predate the multi-seed protocol; the seed-42 LLM run among them was a low outlier), so we read this layer for failure *mechanisms*, not for headline comparisons; those are Tables 6.2 and 6.3. Figure 6.4 shows the class breakdown recomputed on the seed-averaged ensemble predictions of every run; the class profile is the same as in the single-seed analysis described below.

On **WDC Products**, errors are dominated by the *ambiguous variant* class (62–78% of errors across runs): pairs at moderate similarity where the student crossed the decision boundary in the wrong direction, typically product variants differing in one decisive attribute. A representative false positive pairs a Kingston A2000 NVMe SSD in 500 GB against the same drive in 1,000 GB: nearly all tokens agree, the capacity token decides, and the student weights it insufficiently. The second-largest class is dissimilar matches (19–36%): the same product described so differently that token overlap gives the student almost nothing to hold on to. Classic high-similarity non-matches (e.g. a Corsair 570X case in white versus black) are rarer than expected (1–4%) because the class-weighted student already handles many of them. Across training sets the *profile* stays the same; augmentation shifted error *counts*, not error *kinds*, consistent with the Layer 2 finding that no strategy substantially changed what the training data covers.

6. Results

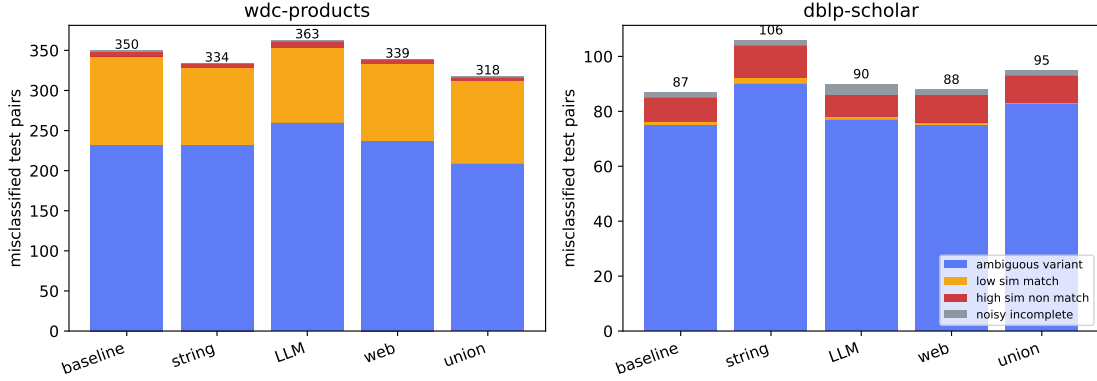


Figure 6.4.: Misclassified test pairs per run, broken down into the four error classes of Section 5.5 (computed on the seed-averaged ensemble predictions). The distribution is dominated by ambiguous variant cases on both datasets, and the profile barely changes across training sets: augmentation shifted error counts, not error kinds.

On **DBLP-Scholar**, students err on 1.6–2.0% of test pairs, and the distribution is nearly invariant across all four training sets: 82–88% are ambiguous variants, plus a consistent handful (7–12 pairs) of a distinctive high-similarity non-match type, namely the same paper title and authors at a conference and in a journal, distinguishable only by venue and year together. A representative example pairs “Querying and Mining Data Streams: You Only Get One Look” at SIGMOD 2002 against SIGMOD Record 2003: identical title, identical authors, different publication. This error type is exactly the hard-negative pattern that DBLP-Scholar’s training data barely contains (Section 6.4), which motivated the targeted mining experiment of Section 7.1. The full per-run breakdown and further examples are given in Appendix D.

6.6.1. Error Overlap Between Students

Do the augmented students fail on the same pairs as the baseline, or on different ones? On WDC Products, the misclassified-pair sets of the baseline and the LLM-augmented student overlap at a Jaccard similarity of 0.47 in the single-seed runs; on the seed-averaged ensemble predictions, where seed-specific noise errors are washed out, the overlap rises to 0.62 (llm_aug), 0.61 (string_aug), and 0.66 (web_aug). In words: roughly half to two thirds of each augmented student’s errors are exactly the baseline’s errors. No strategy carves out and fixes a distinct error class; each mostly re-fails the same hard pairs and trades a handful of errors at the margin. This is the pair-level counterpart of the Layer 1 ties directly: the models make largely the same mistakes, which is consistent with the augmented data adding no new decisive signal.

6.6.2. Evaluation on the Hard Test Subset

The design requirements call for a downstream evaluation restricted to the corner-case pairs of the test set, the pairs where improvement would matter most. Applying the Layer 2 corner-case definition to the test splits yields 526 hard test pairs on WDC Products (455 hard positives, 71 hard negatives) and 177 on DBLP-Scholar (62 and 115). On the seed-averaged predictions, corner-subset F1 on WDC Products is 0.833 for the baseline versus 0.863 (string), 0.857 (llm), and 0.855 (web): all three augmentations sit slightly above the baseline on the hard subset, consistent with their recall gains, though the margins are of the same order as the effects that Layer 1 already judged non-significant. On DBLP-Scholar the corner-subset F1 is 0.770 (baseline), 0.776 (llm), 0.754 (web), and 0.731 (string); string augmentation is clearly worst exactly where precision matters, matching its Layer 1 regression. Two caveats keep this analysis descriptive: the subsets are small, and their class mix differs sharply from the full test split (86% positive on WDC Products against 11% overall), so these F1 values are comparable *across students* but not with the full-test numbers.

6.7. Summary of Findings

1. At the full labeling budget, no expansion strategy improves the student at a level distinguishable from seed noise, on either benchmark (RQ1–RQ3). String augmentation has the best WDC Products mean (+0.016, n.s.) and is the only strategy with a significant effect anywhere: a tiny, consistent *degradation* on DBLP-Scholar. (Section 7.4 shows the full-budget qualifier matters: the LLM additions help significantly at half the budget.)
2. The design targets for the expanded training sets (25% positive, 40–50% corner cases) were not met by any strategy (RQ4). For LLM-based expansion this is structural: entity-disjoint pools on a closed benchmark contain almost no new positives, and the targets were only ever met by the leaked pipeline version.
3. The teacher is not the problem: 97.1% label accuracy (F1 0.862) on adversarially selected pairs, with zero errors on the easier seed pairs.
4. The added slices of the three strategies overlap in zero pairs; they are perfectly complementary, which leaves union training as a natural follow-up.
5. The augmented students make largely the same errors as the baseline: the error sets overlap at Jaccard 0.61–0.66 on the seed-averaged predictions (Section 6.6.1), and no strategy fixes a distinct error class. On the corner-case test subset, all three strategies sit marginally above the baseline on WDC Products and string augmentation is clearly worst on DBLP-Scholar (Section 6.6.2).
6. Single-seed evaluation produced three conclusions (one significant gain, two significant losses) that did not survive the multi-seed protocol; WDC Products seed noise alone spans ± 0.02 F1.

7. Second Iteration: Targeted Improvements

After reviewing the first-round results, a second experimental iteration built on what the analysis had revealed. Four work items resulted: mining the specific hard-negative type that DBLP-Scholar lacks (Section 7.1), fixing the skew of the web slice while injecting deliberately generated hard negatives on WDC Products (Section 7.2), training on unions of the complementary augmentation sets (Section 7.3), and testing whether augmentation helps more when the base training set is artificially shrunk (Section 7.4). Two returned clear negative results, and the union produced the best WDC Products mean without separating from string augmentation alone. But the low-resource ablation (Section 7.4) delivers the thesis’s one unambiguous positive finding: the same LLM-labeled additions that tie the baseline at the full budget help significantly when the base training set is halved, and a volume-matched control pins the gain on the teacher’s labels rather than on data quantity.

7.1. Hard-Negative Mining for DBLP-Scholar

The error analysis identified DBLP-Scholar’s one distinctive hard error type: same title and authors, different publication (conference paper versus its journal version). The Layer 2 analysis showed the training data barely contains such pairs. The obvious intervention is to mine them directly: search the entity tables for record pairs with near-identical titles whose venue *and* year both differ, keeping conference/journal cross-pairs and title duplicates as separate categories.

The mining returned **92** clean hard negatives (42 conference/journal cross-pairs and 50 title duplicates), and this small count is itself the finding: 92 is not a sampling budget but the structural ceiling. DBLP-Scholar simply does not contain many same-title-different-paper cases, which explains both the near-zero hard-negative counts in Layer 2 and, in part, the F1 ceiling. Training with these 92 pairs added (single seed, since the expected effect is bounded by the handful of corresponding test errors) changed nothing: F1 0.9573 versus baseline 0.9579, McNemar $p = 1.0$. A definitive null on a ceiling dataset: even a perfectly targeted repair cannot help when the training data cannot supply more than a few dozen examples of the failure mode, and the failure mode itself accounts for only 7–12 test errors.

7.2. Rebalanced Web Augmentation with Generated Hard Negatives

The web slice’s 81% positive skew (Section 6.4) was the most actionable defect in the first round, and the intervention had two parts. First, *rebalance*: downsample the web positives so the added slice approaches the 1:3 target ratio. Second, *generate hard negatives*: for each of the 300 sampled products, ask the teacher for the names of three real products that are *similar but different* (e.g. the neighboring model in the same product line); retrieve offers for those distractor names via Tavily; have the teacher extract each offer and *verify* it is truly a different product (the verification dropped 142 candidates that turned out to be the same product under another name; the generation step is imperfect in exactly the way one would expect); and pair the original product’s offer with the distractor offer as a labeled hard negative. This produced **1,290** verified hard negatives, and two training variants: a balanced one (added slice $\approx$ 1:3 positive) and a hard-negative-heavy one.

Run	F1 (3 seeds)	Δ F1	McNemar p	Verdict
baseline	0.659 ± 0.015	–	–	–
web_aug (v1)	0.658 ± 0.015	–0.001	0.396	tie
web_v2 balanced	0.643 ± 0.019	–0.016	0.0098	significantly worse
web_v2 hard-neg-heavy	0.631 ± 0.036	–0.029	0.0024	significantly worse*

Table 7.1.: WDC Products: rebalanced web augmentation with generated hard negatives (3 seeds, class-weighted). *Survives Bonferroni correction (one of only two comparisons in the thesis that do, alongside the 50%-budget result of Section 7.4).

The result (Table 7.1) is the opposite of the hypothesis: both variants are significantly *worse* than the baseline, and the harder-negative-heavy variant is worse than the balanced one (-0.029 , $p = 0.0024$, one of only two results in this thesis that survive Bonferroni correction). The error analysis on the seed-averaged predictions locates the damage precisely: false positives *increased* from 216 (baseline) to 259 (balanced) to 283 (hard-neg-heavy), concentrated in the ambiguous-variant class. Adding verified hard negatives made the student over-predict *matches* more.

The mechanism, we argue, is the distribution mismatch anticipated in Section 4.5.4. Every generated hard negative is a pair (clean benchmark record, web-extracted offer), but the test set consists of (clean, clean) pairs. The student learns its decision boundary partly on web-styled text (truncated snippets, marketing phrasing, sparse attributes), and what it learns there does not transfer to benchmark-styled pairs; meanwhile the class-weighted loss over the remaining web positives and the off-distribution negatives pushes the boundary toward over-prediction on the clean data. The negatives themselves were verified to be high quality; the problem is not their labels but their *text*. Together with the fact that LLM augmentation’s clean, in-distribution hard negatives merely produced a tie (Table 6.2), the conclusion of this iteration is: **web augmen-**

tation’s binding problem is the web-versus-benchmark text mismatch, not the positive ratio: fixing the ratio while deepening the mismatch made things strictly worse.

7.3. Training on Unions of the Augmentation Sets

The zero pairwise overlap between the three added slices (Section 6.4) raises a natural question: if each strategy contributes different pairs, does their *union* help where the parts did not? We built union training files on top of the shared base, namely the full union (base + string + LLM + web additions, deduplicated) and all pairwise combinations, and trained the full union under the standard three-seed protocol. The web component is the original slice of Section 4.5, not the v2 variants of Section 7.2, which are significantly harmful on their own and would predictably drag the union down. The combined files are the largest training sets in the thesis: 7,900 pairs (23.6% positive) on WDC Products and 37,187 pairs (18.9% positive) on DBLP-Scholar, i.e. $3.2\times$ and $2.2\times$ the respective base sets.

Dataset	Run	N pairs	F1 (3 seeds)	Δ F1	McNemar p
WDC	baseline	2,500	0.659 ± 0.015	–	–
	string_aug (best single)	5,000	0.675 ± 0.016	+0.016	0.244 (n.s.)
	union all	7,900	0.677 ± 0.015	+0.017	0.0122 (nominal)
DBLP	baseline	17,223	0.9559 ± 0.0019	–	–
	union all	37,187	0.9514 ± 0.0029	–0.005	0.302 (n.s.)

Table 7.2.: Union of all three augmentation slices versus the baseline and the best single strategy (3 seeds; WDC Products runs class-weighted). The WDC Products union has the best mean of any configuration in the thesis but is statistically tied with string augmentation alone ($p = 0.214$); its nominal significance against the baseline does not survive Bonferroni correction.

The results (Table 7.2) are a qualified null. On **WDC Products**, the full union reaches 0.677 ± 0.015 , the best mean of any configuration in this thesis, and the only one whose McNemar test against the baseline drops below 0.05 ($p = 0.0122$). But three facts keep us from calling it a win. First, the p-value is nominal: with roughly a dozen comparisons in the thesis, the Bonferroni threshold is about 0.004, and 0.0122 does not clear it. Second, the union still loses to the baseline on one of three seeds, exactly like string augmentation. Third, and most tellingly, the union is statistically indistinguishable from string augmentation alone ($p = 0.214$, mean difference +0.002): adding the 2,100 LLM-labeled pairs and the 803 web pairs on top of the string copies moved the mean by two thousandths. The complementarity that the zero pair overlap promised does not materialize as complementary *signal*: the union’s effect is the string effect. Given this, we did not train the pairwise combinations; every one of them is bracketed between a component that does nothing on its own and a union that adds

7. Second Iteration: Targeted Improvements

nothing beyond string.

On **DBLP-Scholar**, the union is worse than the baseline on all three seeds (-0.005 , not significant), consistent with its composition: it inherits the reliably harmful string slice (Section 6.2) and dilutes it with two slices that individually did nothing.

The union experiment sharpens the thesis’s central point. Even 5,400 additional WDC Products pairs from three sources with zero mutual overlap (perturbed copies, teacher-labeled hard negatives, and truly new web records) cannot outperform the cheapest of the three on its own. More pairs of the kinds that honest expansion can produce is not the binding constraint; the missing ingredient remains new in-distribution positives, which none of the three sources supplies (Section 9.1).

7.4. Low-Resource Ablation

The comparison with Distiller suggests a specific hypothesis: LLM-labeled data helps most when the base training set is too small to cover the common patterns: Distiller operates at 10% of entities, and its largest gain from teacher labels occurs on its sparsest product dataset. Our WDC Products base set of 2,500 pairs may simply already be past that point. The ablation shrinks the base training set to 50% and 25% (random subsets, preserving the 20% positive rate), combines each reduced base with the unchanged LLM-labeled slice from Section 4.4, and measures whether the augmentation benefit grows as the base shrinks, the regime of Figure 4 of [Steiner and Bizer \(2026\)](#). It does, and this is the one clear positive result of the thesis.

Dataset	Budget	Base only	+ LLM slice	$\Delta F1$	McNemar p
WDC	100% (2,500)	0.659 ± 0.015	0.661 ± 0.020	+0.002	0.356 (n.s.)
	50% (1,250)	0.606 ± 0.024	0.626 ± 0.013	+0.019	0.0002*
	50% + string control	0.606 ± 0.024	0.607 ± 0.023	+0.000	0.128 (n.s.)
	25% (625)	0.000 ± 0.000	0.580 ± 0.046	+0.580	0.0048 (nominal)
DBLP	100% (17,223)	0.9559 ± 0.0019	0.9566 ± 0.0006	+0.001	0.728 (n.s.)
	50% (8,612)	0.9502 ± 0.0032	0.9511 ± 0.0011	+0.001	0.875 (n.s.)
	25% (4,306)	0.9448 ± 0.0031	0.9427 ± 0.0054	-0.002	0.766 (n.s.)

Table 7.3.: Low-resource ablation: F1 of the student trained on a reduced base set, with and without the LLM-labeled slice (3 seeds; WDC Products runs class-weighted). The “string control” row adds a string-augmented slice matched to the LLM slice in size and class balance instead (Section 7.4). *Survives Bonferroni correction. The 100% rows repeat the main results for reference.

Table 7.3 and Figure 7.1 show the outcome. On **WDC Products at 50% budget**, the LLM slice turns 0.606 ± 0.024 into 0.626 ± 0.013 : better on all three seeds, with McNemar $p = 0.0002$ on the seed-averaged predictions, comfortably below the Bonferroni threshold. This is the strongest positive effect in the thesis, and it comes from *exactly the same* 2,100 teacher-labeled pairs that produce a statistical tie at the full budget

7. Second Iteration: Targeted Improvements

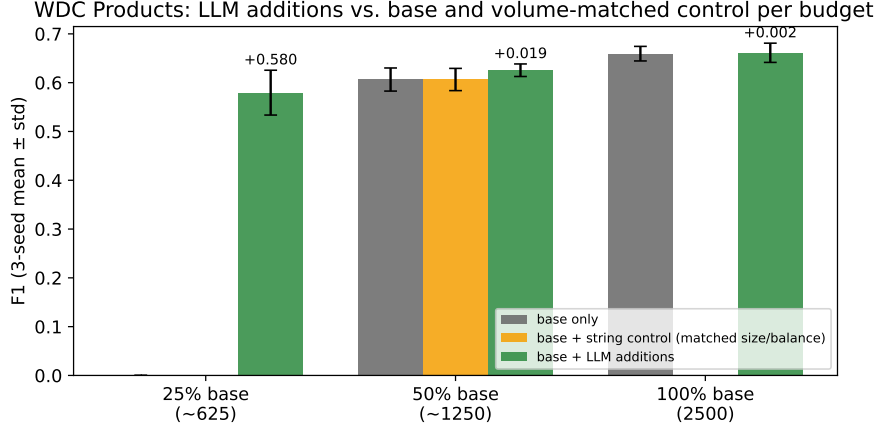


Figure 7.1.: WDC Products: effect of the LLM-labeled additions as a function of the base training budget (3-seed mean $\pm$ std; annotations give the mean difference vs. the base). The same 2,100-pair slice that ties the baseline at the full budget helps significantly at half the budget and rescues an otherwise untrainable set at a quarter. The orange bar at the 50% budget is the volume control: a string-augmented slice of matched size and class balance, which gains nothing over the base.

(+0.002, $p = 0.356$). Nothing about the data changed; only the base against which it is added did.

Signal or Volume? A Matched Control

An obvious objection to the 50% result is that it might reflect data *volume* rather than label *signal*: perhaps any 2,100 additional pairs would have helped a starved base. We tested this with a control arm designed to be as exactly comparable as possible: the halved base is expanded with 2,100 *string-augmented* copies of its own pairs (label-preserving perturbations, Section 4.3), matched to the LLM slice not only in size but also in class balance. Both control and LLM files contain exactly 3,350 pairs with 467 positives (13.9%), so even the class weights of the loss are identical. The only difference between the two arms is where the added pairs’ content and labels come from.

The control changes nothing: 0.607 ± 0.023 versus the base’s 0.606 ± 0.024 (+0.000, $p = 0.128$). Head-to-head, the LLM arm beats the volume control directly (McNemar $p = 0.0216$ on the seed-averaged predictions; ensemble F1 0.627 versus 0.644), though this secondary comparison is nominal under Bonferroni correction; the primary, correction-surviving evidence remains the LLM arm’s $p = 0.0002$ against the base. One nuance is worth recording for completeness: the control’s seed-averaged *ensemble* F1 (0.627) sits slightly above the base’s (0.611) even though its per-seed mean does not move, which is the variance-reduction effect of averaging three models rather than a property of the training data.

7. Second Iteration: Targeted Improvements

Taken together, the 50% row now supports a clean attribution: **the low-resource gain is genuine teacher-label signal, not data volume**. A same-sized, identically balanced addition without new information does nothing; the teacher-labeled addition (new pairings, new hard negatives, and a few new positives) moves the student. This also sharpens the 25% row retroactively: there the base collapses and any volume plausibly helps, which is why we report that row as trainability restoration; at 50%, where the base trains fine, volume alone is demonstrably not the mechanism.

At **25% budget** the effect changes character. The 625-pair base is too small for stable class-weighted fine-tuning: all three seeds collapse to the all-negative predictor ($F1 = 0$). Adding the LLM slice restores a working matcher at 0.580 ± 0.046 . We report this as a *data-volume* effect rather than evidence about label quality (almost any 2,100 additional pairs would presumably prevent the collapse), and note that its McNemar $p = 0.0048$ is only nominal under correction. The honest reading of the 25% row is “LLM expansion restores trainability”, not “LLM expansion adds 0.58 F1 of signal”.

On **DBLP-Scholar**, the ablation is a flat null at every budget ($p = 0.77\text{--}0.88$). Even at a quarter of the data the base student still reaches 0.945 (the dataset is so far inside the ceiling regime that scarcity never bites within the tested range), and the LLM slice, being 0% positive there, has no signal to contribute regardless.

The ablation resolves the thesis’s central tension. The full-budget conclusion (Chapter 6) and Distiller’s optimistic low-resource findings are not in conflict; they sit on two sides of a saturation boundary, and WDC Products crosses that boundary between 1,250 and 2,500 human-labeled pairs. Below it, a positive-starved, hard-negative-heavy teacher-labeled slice carries real marginal signal: the base set undersupplies decisive contrasts, and the additions fill actual gaps. Above it, the same slice is redundant with what the human pairs already teach. This directly confirms the saturation reading of Distiller’s Walmart-Amazon result (Section 8.2.2) and mirrors the shape of the learning curve of [Steiner and Bizer \(2026\)](#): the value of machine-labeled data is a function of where you stand on the budget axis, not a fixed property of the data.

8. Comparison with DistillER and Steiner & Bizer

Two recent works study the same underlying question as this thesis (can an LLM teacher create training data for a small entity matching student?) and reach considerably more optimistic conclusions. This chapter takes both apart in detail and reconciles the difference. It also closes the loop opened in Section 2.8: the F1 numbers collected in Table 2.1 can only be read side by side once budgets, benchmark versions, and task framings are fixed, and fixing them is what this chapter does. Section 8.1 fixes what can and cannot be compared. Sections 8.2 and 8.3 treat DistillER (Zeakis et al. 2026) and Steiner and Bizer (2026) in turn, and Section 8.4 draws the joint picture: the three studies do not contradict each other; they measure different regimes of the same underlying trade-off, and placing them side by side reveals a boundary that none of them shows alone.

8.1. What Is Comparable, and What Is Not

Three axes must be fixed before any numbers are put side by side, because each one silently shifts F1 by more than the effects under discussion.

Task framing: replacement versus expansion. Both comparison works study *replacement*: the teacher labels (or relabels) pairs drawn from the benchmark’s own pair universe, and the student trained on machine labels is compared against a student trained on the same pairs with human labels. The question is “how much quality is lost by swapping the annotator?” This thesis studies *expansion*: the human-labeled training set is kept, and the teacher labels *new* pairs that the benchmark does not contain. The question is “how much quality is gained by adding machine-labeled data?” The distinction sounds procedural but decides what the candidate pool can contain, the central mechanism of Section 8.3.2.

Training budget. DistillER trains on pairs built from roughly 10% of each dataset’s entities, a deliberately low-resource regime. Steiner and Bizer (2026) label at the full benchmark scale, e.g. around 16,000–20,000 WDC Products pairs. This thesis trains on the small WDC Products split (2,500 pairs) plus modest additions. Absolute F1 comparisons across these budgets are meaningless unless made budget-aware, as Section 8.3.4 shows on a concrete case.

Benchmark version. All three works use a WDC Products variant and a DBLP-Scholar-style benchmark, but the variants differ: we and Steiner and Bizer (2026) use the hard 80%-corner-case configuration with the *unseen* (entity-disjoint) test set; DistillER’s product datasets (Abt-Buy, Amazon-Google, Walmart-Amazon) are older Magellan benchmarks with non-disjoint splits. Metric and protocol, at least, agree every-

where: F1 on the match class, mean over three seeds (they use seeds {42, 52, 62}, we use {13, 42, 87}), and we adopt the convention of Steiner and Bizer (2026) that two results are comparable when their ± 1 std intervals overlap.

8.2. Comparison with DistillER

8.2.1. Their Approach

DistillER (Zeakis et al. 2026) distills a Qwen-2.5 32B teacher into small students across eight benchmarks (D2–D9), spanning products, bibliographic data, and other domains. Given a low-resource pair budget (from 10% of entities), the teacher labels the pairs and the student (S-MiniLM or RoBERTa) is fine-tuned on the noisy labels; the control is the same student trained on ground-truth labels for the same pairs. They further compare unsupervised strategies for *selecting* which pairs to label (ranking by blocking similarity emerging as the most reliable, approximately maintaining a 3:1 negative-to-positive ratio), and, for LLM students, supervised fine-tuning on teacher-generated *explanations* rather than bare labels, their best overall variant, though it targets a different student class than ours.

8.2.2. Result Comparison

Table 8.1 places DistillER’s RoBERTa student next to our results on the two shared dimensions: the bibliographic near-ceiling regime and the hard product regime.

Regime	Dataset	RoBERTa+GT labels	RoBERTa+LLM labels
Bibliographic	D9 DBLP-Scholar (DistillER)	0.89	0.86
	DBLP-Scholar (this thesis)	0.9559 ± 0.0019	0.9566 ± 0.0006^a
Products	D2 Abt-Buy (DistillER)	0.78	0.75
	D3 Amazon-Google (DistillER)	0.40	0.41
	D8 Walmart-Amazon (DistillER)	0.44	0.57
	WDC Products (this thesis)	0.659 ± 0.015	0.661 ± 0.020^a
Mean over all 8 (DistillER)		0.66	0.69

Table 8.1.: DistillER (RoBERTa student, Qwen-2.5:32b teacher, $\sim 10\%$ of entities) versus this thesis. ^aOur “LLM labels” column is the *expansion* setting (GT base + LLM-labeled additions), not pure replacement; budgets and benchmark versions also differ (Section 8.1), so trends, not absolute values, are comparable.

Three trends align across the studies, and one deviation is informative.

Bibliographic data: same direction, different absolute level. On DBLP-Scholar-type data, LLM labels do not beat ground truth in either study (DistillER: 0.86 vs. 0.89; ours: statistical tie). Our absolute F1 is far higher (0.956 vs. 0.89) for a budget reason (we train on the full 17,223-pair split, they on a 10% low-resource subset),

which usefully brackets the dataset: even at a tenth of the data, RoBERTa reaches 0.89, and at full data it saturates near 0.956. Both studies conclude that on this benchmark there is little for an LLM teacher to add.

Products are the hard domain, for the same reason. DistillER’s product datasets are their weakest across all methods, and their diagnosis (shared tokens between non-matching entities overwhelming the subtle decisive difference, their example being Cat6 Ethernet cables of 5 m versus 15 m) is precisely the dominant error class of our WDC Products students (the 500 GB versus 1 TB SSD of Section 6.6). Two teachers (Qwen, Claude), two benchmark families, one failure mode: variant-level distinctions are where both teachers and students struggle, which supports treating it as a property of the product-matching task rather than of any single model.

Selection ratios agree. DistillER’s finding that ranking-based selection maintaining roughly 3:1 negatives-to-positives performs best independently supports the 1:3 design target of Section 4.1, and thereby sharpens our negative result: the target itself is well-supported; what our setting lacked was any honest way to *reach* it (Section 6.4).

The Walmart-Amazon deviation points at the interesting regime. D8 is the one case where LLM labels beat ground truth by a wide margin ($0.44 \rightarrow 0.57$). It is also DistillER’s most extreme low-resource case for a product dataset: the ground-truth-trained student is so data-starved that even noisy teacher labels add net signal. Our WDC Products base of 2,500 pairs appears to be past that starvation point: the base set already covers the common patterns, so additional teacher-labeled pairs of the kind our pools could supply added nothing. This reading is testable, and Section 7.4 confirms it: shrinking our base set toward DistillER’s regime makes the augmentation benefit appear: significantly at half the budget ($+0.019$, $p = 0.0002$), and as a rescue from outright training collapse at a quarter.

What does not transfer. DistillER’s central variable is teacher noise (their teacher-student agreement analysis drives the paper); our teacher audit (Section 6.5) shows 2.9% noise, low enough that noise plays almost no role in our results. Conversely, our central variable, the positive content of an entity-disjoint candidate pool, does not arise in their setting at all, because their pairs come from the benchmark’s own pair universe with ground-truth positives included by construction. The two studies fail (and succeed) on orthogonal axes, which is exactly why the joint picture in Section 8.4 is more informative than either alone.

8.3. Comparison with Steiner & Bizer

8.3.1. Their Approach

Steiner and Bizer (2026) ask whether LLM-labeled training data is “fit for use”: they generate candidate pools over entity matching benchmarks (including the same WDC Products 80%-corner-case benchmark with the unseen test set that we use, and DBLP-Scholar), have LLM teachers (GPT-based, among others) label pairs selected by several strategies, train students (Ditto/RoBERTa among them) on the machine-labeled data, and compare against students trained on the original human labels. Their selection

strategies include an active-learning variant (a committee of feature-based classifiers over string-similarity features, querying by disagreement) that our Strategy 2 (Section 4.4) implements step for step. This makes the comparison unusually direct: same student architecture, same benchmark and test set (for WDC Products), same selection algorithm, an LLM teacher on both sides, and one deliberate difference in the candidate pool.

8.3.2. The Pool Difference That Decides Everything

Their pools are built from the benchmark’s pair universe and *recover 99.56–99.92% of the original positive pairs*; the teacher then relabels what the benchmark already knew. Our pipeline *excludes* every known pair (by design, since duplicating or relabeling existing training pairs is not expansion), and what remains after the exclusion is the complement of the benchmark: 5.6% positives on WDC Products, none discoverable on DBLP-Scholar (Section 4.4.2).

This single design choice explains the headline divergence between the two studies. In their setting, active learning selects from a pool with a healthy positive rate, the machine-labeled set has a usable class balance, and the student trained on it lands close to the human-label student: LLM labeling works. In our setting, the same active-learning algorithm, the same features, and a demonstrably accurate teacher (Section 6.5) produce a near-all-negative addition, because the pool has nothing else to offer: LLM *expansion* stalls. Nothing failed in between; the two pools simply contain different things. In distillation terms: replacement lets the teacher transfer knowledge about pairs the benchmark curated, while expansion asks the teacher to create knowledge about pairs nobody curated, and on a closed benchmark, the curatable matches are already taken. We regard making this asymmetry visible and measurable, via the pool-composition numbers, as the main conceptual contribution of the thesis.

8.3.3. Head-to-Head Numbers

Dataset	Quantity	This thesis	Steiner & Bizer
WDC Products	human-label baseline	0.659 $\pm$ 0.015 (2,500 pairs)	0.719 (~16k pairs)
	AL (ML) / llm_aug	0.661 $\pm$ 0.020 (tie, expansion)	0.707 $\pm$ 0.004 (replacement)
	best over strategies	0.675 $\pm$ 0.016 (string, n.s.)	0.722 (AL-Ditto)
DBLP-Scholar	human-label baseline	0.9559 $\pm$ 0.0019	0.956
	AL (ML) / llm_aug	0.9566 $\pm$ 0.0006 (tie)	0.938 $\pm$ 0.004

Table 8.2.: Head-to-head with [Steiner and Bizer \(2026\)](#) (Ditto/RobERTa student). WDC Products: same benchmark and unseen test set, different training budget. “AL (ML)” is their feature-ensemble active-learning variant, algorithmically the same selection as our llm_aug.

Table 8.2 shows the alignment. On **DBLP-Scholar**, where both studies train at full benchmark scale, the human-label baselines are essentially identical (0.9559 vs. 0.956), a useful mutual validation of two independent implementations. Their machine-labeled AL

8. Comparison with DistillER and Steiner & Bizer

variant lands *below* the human baseline (-1.8 F1 points, at 0.938), while our expansion variant ties it ($+0.001$): replacement pays a small noise tax on a saturated dataset, while expansion adds nothing but also breaks nothing. Both studies agree on the substantive point: near the ceiling, an LLM teacher does not improve on human labels.

On **WDC Products** the raw numbers look contradictory (their baseline 0.719 versus ours 0.659) until the budget axis is fixed. Figure 8.1 summarizes the ground-truth-versus-teacher-labels contrast across all three studies in one view.

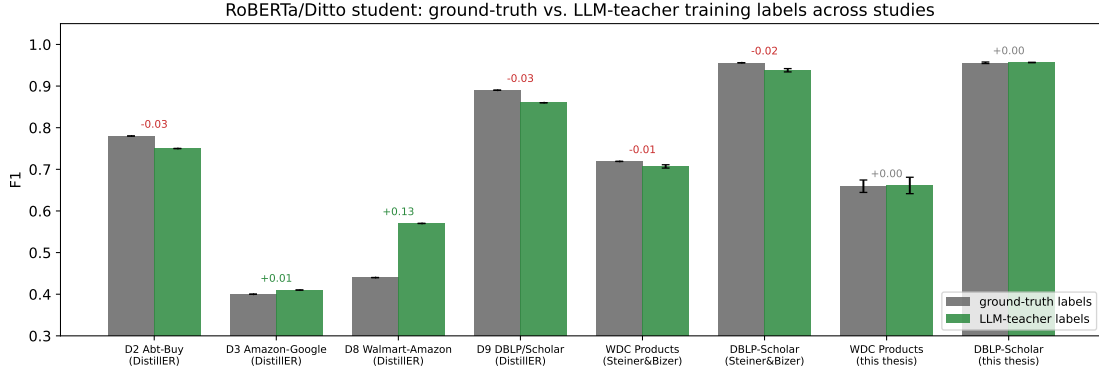


Figure 8.1.: Students trained on ground-truth labels versus LLM-teacher labels across the three studies (RoBERTa/Ditto students; annotations give the delta). DistillER numbers from their Table 8 ($\sim 10\%$ of entities); Steiner & Bizer numbers from their Table 1 (AL-ML variant); our bars are the expansion setting with 3-seed error bars. Budgets and benchmark versions differ between studies (Section 8.1), so the figure compares directions, not absolute levels: teacher labels help only in DistillER’s most data-starved product case (Walmart-Amazon) and are roughly neutral or slightly harmful everywhere else.

8.3.4. A Budget-Aware Reading of the WDC Products Gap

The two baselines differ by exactly one variable: training size. They train on the large split ($\sim 16,000+$ pairs); we train on the small split (2,500 pairs); benchmark, corner-case level, and unseen test set are identical. Their own learning curve (their Figure 4) shows WDC Products F1 rising from roughly 0.50 at 1,000 labeled pairs and only plateauing around 15,000–16,000, crossing 0.719 near the top. Figure 8.2 places our configurations on this budget axis. Two consequences follow.

First, the 0.06 baseline gap is fully explained by budget: 0.659 at 2,500 pairs is *on* their curve, not below it. Second, at a *matched* budget of $\sim 2,500$ labels, their machine-labeled student sits at roughly 0.55–0.62 on that same curve, i.e. *below* our human-label baseline of 0.659. In other words: at our budget, 2,500 human labels beat 2,500 machine labels; machine labeling wins on cost only when it buys volume that humans cannot afford. That is their study, stated budget-aware (machine labels are cheap enough to climb the

8. Comparison with DistillER and Steiner & Bizer

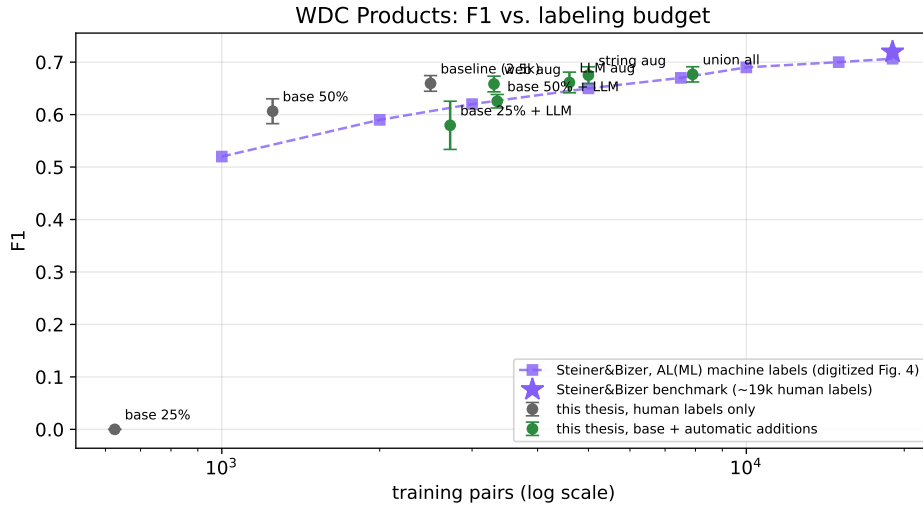


Figure 8.2.: WDC Products F1 as a function of the number of training pairs (log scale). Purple: the machine-label learning curve of [Steiner and Bizer \(2026\)](#), Active learning (ML) variant, digitized from their Figure 4 with the endpoint pinned to their Table 1 value (70.65) and the benchmark line at 71.94; read-off precision is about ± 0.5 –1 F1 per point. The star marks their large-split human-label benchmark. Gray/green with error bars: this thesis’s configurations at their actual training sizes, including the reduced-budget points of Section 7.4. Our human-label baseline at 2,500 pairs lies on their curve; at full budget none of our expansions moves meaningfully up it, while in the reduced-budget region the LLM additions produce visible gains. Growing an adequate training set with expandable pairs is not equivalent to growing it with benchmark-quality labeled pairs, but growing an inadequate one helps.

curve), and it coexists without contradiction with ours: at a fixed, already-labeled base, machine-labeled *expansion* of a closed benchmark adds little, because the addable pairs are the wrong ones. The two studies describe the two ends of the same trade: *scale* (their axis) versus *marginal content* (ours).

8.4. Synthesis

Placing the three studies side by side yields a consistent map of when LLM-teacher data helps a small entity matching student:

1. **Teacher quality is a solved sub-problem on these benchmarks.** DistillER’s Qwen teacher, their GPT-class teachers, and our Claude teacher (97.1% accuracy under adversarial selection) are all accurate enough that label noise is a secondary effect. None of the three studies’ headline outcomes is driven by the teacher being wrong.

2. **What decides the outcome is what the teacher gets to label.** Replacement pools (both comparison works) contain the benchmark’s positives; students trained on their machine labels approach human-label quality. Entity-disjoint expansion pools (this thesis) contain almost no undiscovered positives; the labeled addition is class-degenerate and the student does not move. The candidate pool, not the teacher or the student, is the binding constraint.
3. **Base-set saturation gates the benefit, and we located the gate.** The one large machine-label win across all three studies’ full-budget settings (DistillER’s Walmart-Amazon, +0.13) occurs at extreme data starvation; at our 2,500-pair base the effect is absent, and at full DBLP-Scholar scale everyone’s effects vanish. Our low-resource ablation (Section 7.4) confirms the gate directly on WDC Products: the same LLM additions that tie the baseline at 2,500 pairs help significantly at 1,250 and restore trainability at 625, and a volume-matched control shows the 1,250-pair gain is carried by the teacher’s labels, not by extra data. LLM-teacher data behaves like a remedy for scarcity, not an amplifier of adequacy.
4. **Product-variant distinctions remain the shared frontier.** All three studies bottom out on the same error type (shared-token, single-attribute-difference product pairs) regardless of teacher, student, or labeling regime. Progress here likely requires attribute-aware supervision rather than more pairs of the same kind.

For a practitioner, the map reads: if you have almost no labels, let an LLM teacher label a similarity-ranked pool for you (DistillER’s regime; strong evidence it works). If you can label the benchmark-scale pool, machine labels trade a point or two of F1 for a large cost saving (Steiner & Bizer’s regime). But if you already have a curated training set and hope an LLM will *extend* it over the same entity universe (this thesis’s regime), expect little, *unless that training set is small*: below roughly half our WDC Products base budget, the same additions carry real marginal signal again (Section 7.4). The extendable positives mostly do not exist, and importing them from outside (the web) runs into a text-distribution wall that naive retrieval does not cross; what the additions can supply, hard negatives and variation, only has value where the human pairs do not already supply it.

9. Discussion

9.1. Three Mechanisms Behind the Negative Result

The headline of this thesis, that automatic expansion did not improve the student at the full budget, would be uninteresting if it had a single cause or no cause. It has three, they are separable, and each was established by a different part of the evidence.

Mechanism 1: positive scarcity in entity-disjoint pools. A benchmark is, by construction, an attempt to enumerate the matches among its entities. Whatever match-finding process built the benchmark already harvested the discoverable positives into the labeled splits. An expansion pipeline that (correctly) excludes known pairs and (correctly) restricts itself to training entities therefore fishes in the leftovers: pairs that passed blocking similarity but were never labeled as matches, which are overwhelmingly true negatives. We measured this directly: 5.6% positives in the WDC Products pool, zero discoverable positives on DBLP-Scholar, and consequently a 10.3%- and 0%-positive added slice. No selection algorithm and no teacher can label positives that are not there. The practical formulation: *on a closed benchmark, honest expansion is structurally positive-starved; the accessible new information is almost entirely negative*. Note the qualifier “closed”: the mechanism is about benchmarks whose entity universe is fixed and pre-mined, which is exactly the situation of academic evaluation, and *not* necessarily of production settings where new entities stream in continuously.

Mechanism 2: text-distribution mismatch in web-imported data. Web retrieval gets around Mechanism 1 (it demonstrably injected new positive records, 81% of its slice) and still failed, in the follow-up even significantly (-0.029 F1). The evidence localizes the failure in the text, not the labels: the imported records are web-styled (snippet truncation, marketing language, sparse attributes) while every test pair is benchmark-styled, and training on (clean, web) pairs pushed the student’s boundary in a direction that hurt on (clean, clean) pairs: false positives rose steadily with the amount of web-styled data. Retrieval solves the *content* problem and creates a *style* problem. Any future web-augmentation design must translate retrieved records into the target distribution (or make the student style-invariant) before the added content can pay off.

Mechanism 3: ceiling effects. On DBLP-Scholar every intervention, including a perfectly targeted one (Section 7.1), returned null or a tiny regression. The dataset offers no room: the baseline sits at 0.956 with errors concentrated in a few dozen truly ambiguous pairs, the split is not entity-disjoint (93% of test entities seen in training) and contains 186 text-identical cross-split pairs, and its structure supplies at most 92 examples of its own hardest error type. A ceiling dataset cannot judge between augmentation methods; it can only verify they do no harm. In retrospect, a mid-difficulty

benchmark (e.g. Walmart-Amazon or Abt-Buy) would have been a more informative second dataset, a choice future work can correct cheaply.

The three mechanisms compose into the overall picture: string augmentation adds variation but no information (bounded by design), LLM expansion adds accurate labels over an information-poor pool (bounded by Mechanism 1), and web expansion adds information in the wrong style (bounded by Mechanism 2), while DBLP-Scholar caps everything from above (Mechanism 3). The low-resource ablation adds the boundary condition to all three: these ceilings bind only when the base training set already covers the common patterns. Shrink the base below that coverage point, and the same hard-negative-heavy LLM additions become significantly valuable (Section 7.4): Mechanism 1 limits what expansion can add, but whether that addition *matters* is decided by what the base set lacks.

9.2. Methodological Lessons

Three episodes in this thesis are worth reporting as lessons because each materially changed conclusions, and each guards against a mistake that is easy to make.

Entity leakage in augmentation pipelines is subtle and quantitatively serious. Our first pipeline version excluded known *pairs* but blocked over *all* entities, a design that sounds safe and yet leaked held-out entity text into 35–78% of the added pairs (Section 4.6). The leak did not duplicate any test pair, so no standard check caught it; it surfaced only in an audit that counted entity-level (not pair-level) overlap. The deceptive part is that the leak made results look *better and more interesting*: it supplied exactly the positives whose absence is our Mechanism 1, so the leaked pipeline met the design targets that the honest pipeline cannot meet. Pipelines that manufacture training data should verify entity-level disjointness explicitly, and should treat “the augmented set conveniently meets its targets” as a trigger for suspicion rather than celebration.

Single-seed deltas on small-data fine-tuning are not results. Three of our first-round conclusions (including a $p < 0.001$ improvement) did not survive three-seed averaging (Section 6.3). The WDC Products baseline alone spans ± 0.02 F1 across seeds, the size of a typical published augmentation gain. Multi-seed evaluation is not methodological decoration in this regime; it is the difference between a finding and an artifact.

Paired data demands paired tests. Comparing two models on the same test set with an unpaired two-proportion z -test (as our first analysis did) discards the pairing and, in our case, understated significance; switching to McNemar’s test flipped two DBLP-Scholar conclusions. The general rule (Dietterich 1998) is old, but the error remains common enough to restate.

We also note what these lessons cost: the leakage fix forced regeneration of all teacher-labeled data and retraining of every affected model, and the multi-seed protocol tripled training compute. Both were worth it: every “interesting” result they destroyed was one we would otherwise have reported falsely.

9.3. Limitations

Statistical resolution. Three seeds bound our resolution at roughly ± 0.02 F1 on WDC Products; effects smaller than that are undetectable here. Claims are phrased as “no detectable effect at $n=3$ ”, not “no effect”. Relatedly, McNemar on seed-averaged predictions compares score-ensembles, not per-seed distributions (Section 5.4), and roughly seventeen comparisons were run; only the web-v2 hard-negative degradation and the 50%-budget LLM gain survive Bonferroni; the DBLP string-augmentation regression, the WDC union result, the 25%-budget comparison, and the LLM-versus-control comparison ($p = 0.0216$) are nominal.

Confounded protocol change. The single-seed to multi-seed transition also changed the learning rate ($5 \cdot 10^{-5} \rightarrow 2 \cdot 10^{-5}$), so the overturned first-round conclusions reflect both changes; we cannot cleanly apportion them.

One student, one teacher. All results use **roberta-base** and one teacher model (Claude Sonnet). The teacher-quality audit suggests teacher choice matters little here, but larger students or instruction-tuned students could respond differently to the same data; Distiller’s explanation-tuning results hint that richer supervision may matter more than more pairs.

Budget scale. Our labeling budget (2,100 teacher labels per dataset, 300 retrieval entities) matches the expansion framing of modest additions to an existing set, but leaves untested whether much larger honest expansion (e.g. exhaustively labeling the full 4,722-pair pool, or retrieving for every training entity) would behave differently. Mechanism 1 predicts not, since the pool composition is what it is at any budget; the prediction is testable.

Corner-case definition. Layer 2’s corner-case percentages depend on heuristic Jacard thresholds (0.3/0.4). All cross-strategy comparisons use the same thresholds, so rankings are stable, but the absolute percentages (and the exact size of the miss against the 40–50% target) shift with the threshold choice.

Two benchmarks. The mechanisms were established on one hard product benchmark and one ceiling bibliographic benchmark. Mechanism 1 in particular should generalize to any closed, well-mined benchmark, but breadth of the kind Distiller has (eight datasets) is beyond this thesis’s compute.

10. Conclusions

This thesis asked whether an existing entity matching training set can be expanded automatically by string augmentation, by LLM-teacher labeling of actively selected new pairs, or by LLM-labeled web retrieval. The answer turned out to depend on where you stand: a careful no at the full labeling budget, and a clear yes once labels are scarce, together with an account of the mechanisms behind both halves.

Across two benchmarks and a three-seed protocol with paired significance tests, no strategy improved a RoBERTa student over its human-labeled baseline (RQ1–RQ3): string augmentation came closest on WDC Products (+0.016 mean F1, not significant) while slightly but consistently harming DBLP-Scholar; LLM-based and web-based expansion tied their baselines everywhere; and even the union of all three mutually non-overlapping additions only reproduced string augmentation’s result (+0.017, statistically tied with string alone). The expanded training sets missed their design targets for positive ratio and corner-case share (RQ4), and the miss is structural rather than accidental: once expansion is honestly entity-disjoint, the candidate pool of a closed benchmark contains almost no undiscovered positives (5.6% on WDC Products, none on DBLP-Scholar). The LLM teacher itself performed excellently (97.1% label accuracy against cluster ground truth, with errors concentrated exactly where uncertainty sampling sent it), so the bottleneck is what there is to label, not how well it is labeled. Web retrieval, the one strategy that truly imports new positives, stumbled over a second wall: retrieved records are written in web style, benchmark test pairs in benchmark style, and a targeted second-iteration experiment showed that deepening the web-styled data (even with verified, high-quality generated hard negatives) makes the student significantly worse, pinning the failure on text distribution rather than label balance.

The low-resource ablation completes the picture and bounds the negative result. Cutting the WDC Products base set to half turns the LLM-labeled additions from a statistical tie into a significant gain (0.606 to 0.626 F1, $p = 0.0002$, one of only two Bonferroni-surviving effects in the thesis), and a control that instead adds a string-augmented slice of identical size and class balance gains nothing (+0.000): the gain is teacher-label signal, not data volume. Cutting the base to a quarter makes the additions the difference between a collapsed model and a working one. The same 2,100 teacher-labeled pairs are worthless at 2,500 human pairs and valuable at 1,250, so the operative question for automatic expansion is not “does it work?” but “is the existing training set already past its coverage point?”. On our benchmarks that point lies between 1,250 and 2,500 pairs for WDC Products, while DBLP-Scholar never leaves the saturated regime within the tested range.

Set against Distiller and [Steiner and Bizer \(2026\)](#) (RQ5), these results are not a contradiction but a boundary, one our ablation locates empirically. Their replacement

10. Conclusions

setting (the teacher relabels the benchmark’s own pairs, positives included by construction) works, and works for the reason our audit confirms: modern LLM teachers are accurate annotators. Our expansion setting starves the same machinery of positives at full budget, yet recovers their optimistic conclusion exactly where their settings live: in the scarce-label regime. Together the three studies say: LLM teachers remedy label *scarcity*; they do not conjure pair *content* that a closed benchmark does not contain, and whether that limitation bites depends entirely on how much labeled data you already have.

10.1. Future Work

The experimental program of the thesis is complete; the natural next step within it is to resolve the coverage point more finely. The ablation brackets WDC Products’ saturation boundary between 1,250 and 2,500 human-labeled pairs with two budget levels; a denser budget grid (and a second mid-difficulty product benchmark) would turn the bracket into a curve and show whether the crossover behaves like the learning-curve knee of [Steiner and Bizer \(2026\)](#).

Beyond that, three directions follow from the mechanisms. First, *style transfer for retrieved data*: web retrieval solved the content problem, so normalizing retrieved records into the benchmark’s textual distribution (an LLM rewriting step, or training-time style augmentation) attacks the one wall it hit. Second, *a matched replacement arm*: re-labeling our 2,500 training pairs with the same Claude teacher and training the same student would replicate [Steiner and Bizer \(2026\)](#) inside this exact setup, making the augmentation-versus-replacement contrast an in-house, same-budget measurement instead of a cross-paper one. Third, *attribute-aware supervision*: every study in Chapter 8 bottoms out on single-attribute product variants, suggesting that the next gains lie not in more pairs but in supervision that names the decisive attribute: teacher-generated explanations, contrastive pairs that isolate one attribute, or auxiliary attribute-comparison objectives.

Finally, the negative results here were made trustworthy by three practices (entity-level leakage audits, multi-seed evaluation, and paired testing) whose absence would have produced a very different-looking, and wrong, thesis. We would like to see them become as standard for data-centric entity matching studies as blocking recall checks already are.

Bibliography

- Barlaug, N. and J. A. Gulla (2021). Neural networks for entity matching: A survey. *ACM Transactions on Knowledge Discovery from Data* 15(3), 1–37.
- Breiman, L. (2001). Random forests. *Machine Learning* 45(1), 5–32.
- Brunner, U. and K. Stockinger (2020). Entity matching with transformer architectures — a step forward in data integration. In *Proceedings of the 23rd International Conference on Extending Database Technology (EDBT)*, pp. 463–473.
- Chen, T. and C. Guestrin (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- Christen, P. (2012). *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Berlin, Heidelberg: Springer.
- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Hillsdale, NJ: Lawrence Erlbaum Associates.
- Devlin, J., M.-W. Chang, K. Lee, and K. Toutanova (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pp. 4171–4186.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation* 10(7), 1895–1923.
- Dunn, O. J. (1961). Multiple comparisons among means. *Journal of the American Statistical Association* 56(293), 52–64.
- Ebraheem, M., S. Thirumuruganathan, S. Joty, M. Ouzzani, and N. Tang (2018). Distributed representations of tuples for entity resolution. *Proceedings of the VLDB Endowment* 11(11), 1454–1467.
- Efron, B. and R. J. Tibshirani (1994). *An Introduction to the Bootstrap*. Boca Raton: Chapman & Hall/CRC.
- Elmagarmid, A. K., P. G. Ipeirotis, and V. S. Verykios (2007). Duplicate record detection: A survey. *IEEE Transactions on Knowledge and Data Engineering* 19(1), 1–16.

Bibliography

- Fellegi, I. P. and A. B. Sunter (1969). A theory for record linkage. *Journal of the American Statistical Association* 64(328), 1183–1210.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* 29(5), 1189–1232.
- Gou, J., B. Yu, S. J. Maybank, and D. Tao (2021). Knowledge distillation: A survey. *International Journal of Computer Vision* 129(6), 1789–1819.
- Hinton, G., O. Vinyals, and J. Dean (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Kasai, J., K. Qian, S. Gurajada, Y. Li, and L. Popa (2019). Low-resource deep entity resolution with transfer and active learning. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 5851–5861.
- Konda, P., S. Das, P. Suganthan G. C., A. Doan, A. Ardalan, J. R. Ballard, H. Li, F. Panahi, H. Zhang, J. Naughton, S. Prasad, G. Krishnan, R. Deep, and V. Raghavendra (2016). Magellan: Toward building entity matching management systems. *Proceedings of the VLDB Endowment* 9(12), 1197–1208.
- Köpcke, H., A. Thor, and E. Rahm (2010). Evaluation of entity resolution approaches on real-world match problems. *Proceedings of the VLDB Endowment* 3(1), 484–493.
- Li, Y., J. Li, Y. Suhara, A. Doan, and W.-C. Tan (2020). Deep entity matching with pre-trained language models. *Proceedings of the VLDB Endowment* 14(1), 50–60.
- Liu, Y., M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Loshchilov, I. and F. Hutter (2019). Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12(2), 153–157.
- Meduri, V. V., L. Popa, P. Sen, and M. Sarwat (2020). A comprehensive benchmark framework for active learning methods in entity matching. *Proceedings of the 2020 International Conference on Management of Data (SIGMOD)*, 1133–1147.
- Miao, Z., Y. Li, and X. Wang (2021). Rotom: A meta-learned data augmentation framework for entity matching, data cleaning, text classification, and beyond. *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 1303–1316.
- Mudgal, S., H. Li, T. Rekatsinas, A. Doan, Y. Park, G. Krishnan, R. Deep, E. Arcaute, and V. Raghavendra (2018). Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pp. 19–34.

Bibliography

- Narayan, A., I. Chami, L. Orr, and C. Ré (2022). Can foundation models wrangle your data? *Proceedings of the VLDB Endowment* 16(4), 738–746.
- Newcombe, H. B., J. M. Kennedy, S. J. Axford, and A. P. James (1959). Automatic linkage of vital records. *Science* 130(3381), 954–959.
- Papadakis, G., D. Skoutas, E. Thanos, and T. Palpanas (2020). Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys* 53(2), 1–42.
- Peeters, R. and C. Bizer (2023). Using ChatGPT for entity matching. In *New Trends in Database and Information Systems (ADBIS 2023 Short Papers)*, pp. 221–230.
- Peeters, R., R. C. Der, and C. Bizer (2024). WDC products: A multi-dimensional entity matching benchmark. *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*, 22–33.
- Peeters, R., A. Steiner, and C. Bizer (2025). Entity matching using large language models. *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*, 529–541.
- Primpeli, A. and C. Bizer (2021). Graph-boosted active learning for multi-source entity resolution. In *Proceedings of the 20th International Semantic Web Conference (ISWC)*, pp. 182–199.
- Primpeli, A., R. Peeters, and C. Bizer (2019). The WDC training dataset and gold standard for large-scale product matching. In *Companion Proceedings of the 2019 World Wide Web Conference*, pp. 381–386.
- Reimers, N. and I. Gurevych (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992.
- Sanh, V., L. Debut, J. Chaumond, and T. Wolf (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.
- Settles, B. (2009). Active learning literature survey. Technical Report 1648, University of Wisconsin–Madison, Department of Computer Sciences.
- Shorten, C., T. M. Khoshgoftaar, and B. Furht (2021). Text data augmentation for deep learning. *Journal of Big Data* 8(1), 101.
- Steiner, A. and C. Bizer (2026). Labeling training data for entity matching using large language models. *arXiv preprint arXiv:2606.28823*.
- Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 5998–6008.

Bibliography

- Wang, W., F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, pp. 5776–5788.
- Wei, J. and K. Zou (2019). EDA: Easy data augmentation techniques for boosting performance on text classification tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pp. 6382–6388.
- Wolf, T., L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45.
- Zeakis, A., G. Papadakis, D. Skoutas, and M. Koubarakis (2026). DistillER: Knowledge distillation in entity resolution with large language models. *arXiv preprint arXiv:2602.05452*.

A. Teacher Prompts

This appendix reproduces the prompts used with the teacher model (claude-sonnet-4-6) in the LLM-based and web-retrieval pipelines. All prompts require a strict JSON answer; malformed responses were retried up to three times. {left} and {right} denote the serialized records of a pair.

A.1. Pair Labeling (WDC Products)

```
You are labeling pairs of e-commerce product listings for entity
matching.
Decide if LEFT and RIGHT refer to the exact same product (same model,
variant, and specs, not just the same product line).
Pay close attention to: color, storage/memory capacity,
size/dimensions, bundle contents, edition/version, and model number.

LEFT: {left}
RIGHT: {right}

Reply with valid JSON only, no markdown, no extra text:
{"label": 1, "reasoning": "one sentence"} ← same product
{"label": 0, "reasoning": "one sentence"} ← different products
```

A.2. Pair Labeling (DBLP-Scholar)

```
You are labeling pairs of academic paper records for entity matching.
Decide if LEFT and RIGHT refer to the exact same paper (same title,
same authors, same publication).

LEFT: {left}
RIGHT: {right}

Reply with valid JSON only, no markdown, no extra text:
{"label": 1, "reasoning": "one sentence"} ← same paper
{"label": 0, "reasoning": "one sentence"} ← different papers
```

A.3. Web Snippet Extraction and Labeling

The web pipeline (Section 4.5.3) combines relevance filtering, schema extraction, and labeling in one prompt. The product version is shown; the paper version substitutes the record type, schema, and match definition, and omits the attribute-attention line.

```
You are extracting structured records from web search results for
entity matching.
```

A. Teacher Prompts

ORIGINAL RECORD (e-commerce product listing):
{original_record}

WEB SEARCH RESULT:

Title: {web_title}

URL: {web_url}

Content: {web_content}

Task:

1. Decide if the web search result describes a SPECIFIC e-commerce product listing whose fields (brand, title, description, price, priceCurrency) can be extracted from the title/content above. If it is irrelevant (e.g. a category/listing page, blog post, search results page, or does not describe a specific record), respond with {"relevant": false} and nothing else.

2. If relevant, extract a record with exactly these fields: brand, title, description, price, priceCurrency (use "" for any field you cannot determine from the text).

3. Decide whether the extracted record refers to the exact same product (same model, variant, and specs, not just the same product line) as the ORIGINAL RECORD (label 1) or not (label 0).

Pay close attention to: color, storage/memory capacity, size/dimensions, bundle contents, edition/version, and model number.

Reply with valid JSON only, no markdown, no extra text.

A.4. Hard-Negative Generation and Verification

The second-iteration experiment of Section 7.2 uses two further prompts. The *generation* prompt presents a serialized product record and asks the teacher to list three names of “SIMILAR BUT DIFFERENT real products that are easy to confuse with it” (e.g. neighboring models of the same product line), returned as a JSON list. The *verification* prompt has the same structure as the extraction prompt of Appendix A.3, but its labeling instruction is inverted: after extracting the distractor offer, the teacher must confirm that it is NOT the same product as the original record; candidates the teacher judges to be the same product are discarded (142 of 1,432 generated candidates were dropped this way).

B. Hyperparameters and Reproducibility

Parameter	Value
Student model	roberta-base (125M parameters)
Sequence length	256 tokens (pair encoding, truncation)
Epochs	up to 10, early stopping patience 3 (validation F1)
Batch size	8, gradient accumulation 4 (effective 32)
Learning rate	$2 \cdot 10^{-5}$ (final protocol; initial single-seed runs: $5 \cdot 10^{-5}$)
Optimizer	AdamW, weight decay 0.01, warmup 100 steps
Class weighting	inverse-frequency, WDC Products runs only
Seeds	13, 42, 87
Model selection	best validation F1 checkpoint
Decision rule	argmax (0.5 threshold on match probability)
Embedding model (blocking)	all-MiniLM-L6-v2 , cosine, $k = 20$
AL committee	RandomForest, XGBoost, GradientBoosting, LogisticRegression
AL features	token Jaccard, char-bigram Jaccard, Jaro-Winkler, Levenshtein ratio, token-set ratio, embedding cosine
AL schedule	100 seed labels + 20 iterations $\times$ 100 labels
Teacher model	claude-sonnet-4-6 , max 128 output tokens (labeling)
Web search	Tavily, basic depth, 5 results/query, top 3 used
Web sampling	150 entities/side, brand- or venue-stratified, 15% bucket cap
Significance	McNemar exact binomial on seed-averaged predictions; paired bootstrap on F1, 10,000 resamples; Cohen’s h

Table B.1.: Complete configuration of all experiments.

Table B.1 lists every setting needed to reproduce the experiments. Per-pair predictions and match scores of every run are stored alongside the aggregated results, so all reported metrics, significance tests, and error analyses can be recomputed from the saved artifacts without retraining. The augmented training files themselves are deterministic given the stored teacher labels; the teacher calls are the only non-reproducible component (hosted model), which is why all teacher outputs, including reasoning sentences, were persisted at labeling time. The complete implementation of all experiments, including the data preparation, augmentation pipelines, training and evaluation scripts, and analysis code, is publicly available in the accompanying GitHub repository.¹

¹<https://github.com/abdmomin/thesis-project>

C. Additional Tables

C.1. Threshold Calibration on WDC Products

Operating point	Threshold	F1	Δ F1
Default (argmax)	0.500	0.641	–
Max-F1 on validation	0.444	0.632	−0.009
Matched to test prior ($\pi = 0.11$)	0.986	0.565	−0.076
Max-F1 on test (oracle upper bound)	0.843	0.659	+0.018

Table C.1.: Decision-threshold calibration of the class-weighted WDC Products baseline (single seed). No selectable operating point beats the default; the oracle bound caps threshold gains at +0.018 F1.

C.2. Per-Seed F1 Values

Dataset	Run	Seed 13	Seed 42	Seed 87	Mean $\pm$ std
WDC	baseline	0.6765	0.6488	0.6530	0.659 $\pm$ 0.015
	string_aug	0.6598	0.6737	0.6918	0.675 $\pm$ 0.016
	llm_aug	0.6431	0.6585	0.6822	0.661 $\pm$ 0.020
	web_aug	0.6444	0.6741	0.6566	0.658 $\pm$ 0.015
	web_v2 balanced	0.6339	0.6649	0.6312	0.643 $\pm$ 0.019
	web_v2 hard-neg	0.6528	0.5897	0.6494	0.631 $\pm$ 0.036
DBLP	baseline	0.9544	0.9554	0.9580	0.9559 $\pm$ 0.0019
	string_aug	0.9490	0.9499	0.9460	0.9483 $\pm$ 0.0020
	llm_aug	0.9572	0.9561	0.9565	0.9566 $\pm$ 0.0006
	web_aug	0.9566	0.9572	0.9562	0.9567 $\pm$ 0.0005

Table C.2.: Per-seed test F1 of every multi-seed run. Note the spread of the WDC Products baseline (0.028 between best and worst seed) relative to the size of the augmentation effects.

C.3. Decisive-Attribute Coverage (WDC Products)

Training set	color	mem./storage	size	bundle	edition	model no.
baseline	36.4	40.5	36.8	13.2	20.7	35.0
string_aug	35.5	39.9	36.4	13.0	20.3	34.1
llm_aug	36.9	35.2	35.7	13.0	20.7	31.8
web_aug	35.0	37.9	34.9	13.0	19.4	33.2

Table C.3.: Share of pairs (%) in which either record mentions each decisive attribute type. All strategies broadly preserve the baseline’s coverage; none expands coverage of underrepresented attributes such as bundle contents.

C.4. Candidate Pool Composition (LLM Pipeline)

	WDC Products	DBLP-Scholar
Entities (left / right)	2,896 / 2,891	2,616 / 64,263
Raw blocking candidates ($k=20$)	57,920	52,320
Known pairs removed	3,171	10,925
Non-train-entity pairs removed	47,236	22,554
Final pool	4,722	18,841
True matches in pool (ground truth)	266 (5.6%)	≈ 0 (none found)
Pool cosine similarity (mean)	0.83	0.75
Teacher labels spent	2,100	2,100
Positive teacher labels	217 (10.3%)	0 (0.0%)

Table C.4.: Composition of the entity-disjoint candidate pools and the resulting teacher-label balance. For DBLP-Scholar the true-match count in the pool cannot be computed exactly (no cluster ground truth); the seeding and active-learning stages discovered none in 2,100 attempts.

D. Representative Error Examples

The following examples are drawn from the stored per-pair predictions and illustrate the main error classes of Section 6.6. Records are abbreviated; similarity is token-level Jaccard.

High-similarity non-match (WDC Products). Color variant confusion:

LEFT: Corsair -- CORSAIR Crystal 570X RGB Tempered Glass, Premium ATX Mid Tower Case, White
RIGHT: Corsair -- CORSAIR Crystal 570X RGB Tempered Glass, Premium ATX Mid Tower Case, Black
true = non-match, predicted = match, sim = 0.70. The two descriptions differ only in the color token.

Ambiguous variant (WDC Products). Capacity variant confusion, the dominant class:

LEFT: Kingston -- Kingston A2000 NVMe PCIe SSD 500GB
RIGHT: Kingston -- Kingston A2000 NVMe PCIe SSD 1000GB
true = non-match, predicted = match, sim = 0.52. Same product line, one decisive attribute (capacity) differs.

High-similarity non-match (DBLP-Scholar). Conference paper versus its journal version:

LEFT: Querying and Mining Data Streams: You Only Get One Look -- J. Gehrke, F. Korn, S. Muthukrishnan -- SIGMOD -- 2002
RIGHT: Querying and Mining Data Streams: You Only Get One Look -- J. Gehrke, F. Korn, S. Muthukrishnan -- SIGMOD Record -- 2003
true = non-match, predicted = match, sim = 0.64. Identical title and authors; only venue and year jointly resolve the pair. This is the error type targeted (unsuccessfully, for lack of training examples) in Section 7.1.

The complete error listings for all runs, including the per-class counts underlying Section 6.6 and further examples, are contained in the stored analysis outputs of the project repository.

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Masterarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools			
Tool	Purpose	Where?	Useful?
Claude	LLM teacher for pair labeling and extraction	Ch. 4–7, data pipelines	++
Claude	Code assistance for experiment scripts	<code>src/</code> , notebooks	++
Claude	Text drafting and rephrasing	Throughout	++
Tavily	Web search API for retrieval augmentation	Ch. 4–7	+

Unterschrift
Mannheim, den 31. Juli 2026